# Teoría de PSyD

Daniel Báscones García

6 de noviembre de 2026

## Índice

**1. Del código de alto nivel a la memoria física** **3**
1.1. La estructura del hardware - dispositivos mapeados en memoria . . . 3
1.2. El acceso a memoria - punteros y direcciones . . . 4
1.3. Las operaciones bit a bit - manipulación de registros . . . 5
1.4. Definiciones vs variables - el espacio utilizado . . . 6

**2. Estructuras y definiciones de hardware** **8**
2.1. La palabra clave `volatile` . . . 8
2.2. Abstracción de Hardware con `struct` . . . 8
2.3. Utilización de `union` y campos de bits . . . 9
2.4. Registros mayores al tamaño de palabra . . . 10
2.5. Palabra clave `static` . . . 10
2.5.1. Funciones `static inline` . . . 11

**3. El enlazado de programas e inicialización** **13**
3.1. El *linker script* . . . 13
3.2. Código de inicio y salto a `main` . . . 14
3.3. La tabla de vectores . . . 15
3.3.1. El modificador *weak* . . . 16

**4. Interrupciones, sincronización y *callbacks*** **17**
4.1. Barreras de memoria . . . 18
4.2. Punteros a función y Callbacks . . . 19

**5. Los periféricos del procesador** **21**
5.1. El bus AXI . . . 21
5.1.1. *Handshake* o sincronización . . . 23
5.2. El bus AHB y APB . . . 23
5.3. El puente entre bus y registro . . . 23
5.4. La interconexión con diferentes memorias . . . 24
5.4.1. Memorias acopladas mediante ITCM y DTCM . . . 24
5.4.2. Controladores de memoria externa (FMC, QSPI, Flash...) . . . 24
5.5. Ejemplos de periféricos . . . 25
5.6. Activación de periféricos . . . 25

**6. Temporización** **26**
6.1. Osciladores . . . 26
6.2. Árbol de reloj . . . 27
6.3. Ticker del sistema . . . 27
6.4. Temporizadores . . . 29

6.5. RTC (*Real-Time Clock*) . . . . . 30

**7. Periféricos y buses externos** **31**
7.1. I2C . . . . . 31
7.1.1. Funcionamiento del bus I2C - capa física . . . . . 32
7.1.2. Direccionamiento en I2C - capa lógica . . . . . 32
7.1.3. Líneas auxiliares a I2C . . . . . 34
7.2. SPI . . . . . 34
7.2.1. Funcionamiento del bus SPI - capa física . . . . . 35
7.2.2. Funcionamiento del bus SPI - capa lógica . . . . . 35
7.2.3. Topologías de conexión . . . . . 36
7.3. Dispositivos externos mapeados en memoria (FMC) . . . . . 36
7.3.1. Acceso por software . . . . . 37
7.4. Comparativa . . . . . 37

**8. Multitarea** **38**
8.1. Arquitecturas multitarea cooperativa . . . . . 38
8.1.1. Super-loop . . . . . 38
8.1.2. Foreground/Background . . . . . 39
8.1.3. Cola de funciones . . . . . 39
8.2. Arquitecturas RTOS . . . . . 40
8.3. Correción en aplicaciones multihebra . . . . . 42
8.4. Sistemas guiados por tiempo . . . . . 43
8.4.1. Primera aproximación . . . . . 43
8.4.2. Segunda aproximación . . . . . 44
8.4.3. Tercera aproximación . . . . . 45
8.4.4. Polling con espera en tareas periódicas . . . . . 45
8.5. Planificadores cooperativos . . . . . 46
8.5.1. Bloqueos . . . . . 48

# 1. Del código de alto nivel a la memoria física

Los lenguajes de alto nivel son cada vez más abstractos. En un origen, la programación se hacía directamente en ensamblador, atacando directamente al conjunto de instrucciones del propio procesador. Con la creación y avance de los compiladores, estos se sustituyeron por lenguajes algo más abstractos como `COBOL`, `Pascal`, o en última instancia `C`, incluyendo conceptos como funciones o tipos, e incluso multitarea. Posteriormente, se introdujeron lenguajes más complejos como `C++`, que añade clases, orientación a objetos, plantillas, y multitud de comodidades en versiones más recientes como programación funcional, pero aún con posibilidades cercanas al hardware. Hoy en día, la mayor parte de los programadores trabajan en lenguajes de aún más alto nivel, como `Java` o `Python`, que ni siquiera se ejecutan en su mayor parte directamente sobre hardware, sino sobre máquina virtual, lo cual permite estilos de programación muy flexibles donde el concepto de tipos o funciones se diluye.

Esta evolución la permite, en parte, el *stack* tecnológico que vive debajo: Un procesador complejo, sobre el que se ejecuta un sistema operativo, que a su vez tiene un intérprete instalado al que ofrece interfaces de control, sobre el cual finalmente se ejecuta el lenguaje de nuestra elección.

Sin embargo, sigue habiendo nichos donde tener toda esta pila de herramientas no es posible: uno de ellos son los sistemas empotrados.

Un sistema empotrado, aunque su definición sea en ocasiones difusa, suele tener capacidades muy reducidas, y estar diseñado para funciones muy específicas. Dada la simplicidad de su diseño, los procesadores que incluyen no soportarían (o serían demasiado lentos) un sistema con la complejidad como para ejecutar todos estos lenguajes modernos. Por tanto, aún se siguen utilizando lenguajes más simples y con características más cercanas al hardware. En concreto, el lenguaje rey en este ámbito sigue siendo `C`. Aún y con todo, `C` sigue siendo un lenguaje de alto nivel, y conviene entender cómo nos va a permitir trabajar directamente con el *hardware*.

## 1.1. La estructura del hardware - dispositivos mapeados en memoria

Un procesador, por sí solo, no es más que una máquina de ejecutar instrucciones. Según el conjunto de instrucciones con que diseñemos nuestro procesador, le daremos más o menos capacidades. Aún así, muchas de las capacidades no residen en el propio núcleo en sí, sino en periféricos asociados que liberan de trabajo al *core* ejecutor de tareas.

En las arquitecturas de microcontroladores modernas (como la familia ARM Cortex-M), el procesador utiliza, para comunicarse con dichos periféricos, un esquema denominado *Memory-Mapped I/O* (MMIO). Así, para comunciarse con los periféricos del procesador, no se utilizan instrucciones específicas. En su lugar, todos los periféricos comparten el mismo espacio de direccionamiento de 32 bits ($4GB$ de direcciones) junto con la memoria de programa (Flash) y la memoria de datos (RAM).

- **Memoria Flash** (`0x08000000`): Almacena el código compilado, constantes, e información persistente. Es memoria no volátil, que se mantiene tras un apagado o reinicio del sistema.
- **Memoria SRAM** (`0x20000000`): Almacena las variables globales, la pila (*stack*) y el montón (*heap*). Es memoria volátil de acceso rápido, que se reinicia en cada inicio del sistema.

- **Periféricos** (`0x40000000 - 0x50000000` y otros): Colección de registros físicos mapeados directamente en direcciones de memoria fijas.

Desde la perspectiva del procesador, escribir en la dirección `0x40020000` no difiere sintácticamente de escribir en una variable almacenada en la RAM. Sin embargo, la circuitería del bus redirige esa operación al controlador del periférico correspondiente (por ejemplo, puerto de entrada/salida de propósito general), provocando un cambio de estado ajeno al procesador, como por ejemplo el encendido de un LED.

## 1.2. El acceso a memoria - punteros y direcciones

Para interactuar con las direcciones físicas descritas en el modelo MMIO desde el lenguaje `C`, se emplean punteros. En `C`, debemos trabajar manualmente con valores numéricos (representando dichas direcciones) para conseguir acceder a las mismas. Un puntero se describe de la siguiente manera:

```c
#include <stdint.h>

uint32_t * puntero;
```

**NOTA:** Es fundamental utilizar tipos de datos con ancho de bits garantizado mediante la librería de la cabecera estándar `<stdint.h>` (como `uint32_t, uint16_t, uint8_t`), evitando tipos ambiguos como `int` o `long` cuyo tamaño puede variar según arquitectura y compilador.

Un puntero no es más que una dirección de memoria. En este caso, `uint32_t *` representa "dirección de memoria donde hay almacenado un `uint32_t`". Si queremos cambiar ese valor, guardado en dicha dirección de memoria, haremos:

```c
*puntero = 20;
```

Y, si queremos cambiar la dirección de memoria a la que apuntamos, haremos:

```c
puntero = (uint32_t *) 0x40020000U;
```

Así, conociendo la dirección de memoria a través de la cual podemos acceder a un periférico (por ejemplo para encender un LED), podemos escribir en ese preciso lugar de la siguiente manera:

```c
#include <stdint.h>

// Macro para poder cambiar fácilmente la dirección
#define DIRECCION_OBJETIVO (0x40020000U)

void encender_led(void) {
    // 1) creación de nuestro puntero (indicamos el tipo del mismo)
    uint32_t * p_led = (uint32_t *) DIRECCION_OBJETIVO;

    // 2) Escribimos sobre esa dirección de memoria
    *p_led = 0x00000001;
}
```

En la práctica, se suelen combinar ambas operaciones en una única macro para facilitar la escritura de código:

```c
#define REGISTRO_LED  (*(volatile uint32_t *) 0x40020000U)
// Ahora podemos utilizar la macro como si fuera una variable
REGISTRO_LED = 0x00000001;
```

Si aún quedan dudas, consulta la Figura 1 para verlo en forma de diagrama.

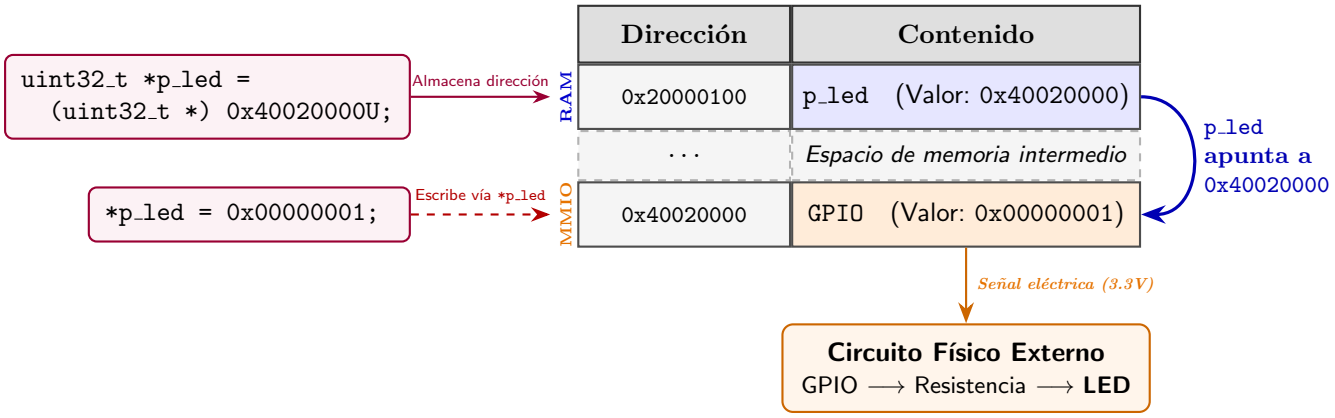



Figura 1: Esquema de las dos direcciones de memoria involucradas con un puntero: la dirección a la que apunta, y el valor apuntado.

**NOTA:** Si hablamos de punteros, no podemos dejar de lado (aunque no se utilice habitualmente) el operador `&`. Este operador sirve para darnos la *dirección* de una variable. Así, podremos establecer punteros a direcciones concretas donde tenemos almacenados datos:

```c
uint32_t    a;
uint32_t* ptr;
 ptr = &a; //guardamos dirección de 'a' en el puntero
*ptr = 20; //modificamos 'a' directamente
```

## 1.3. Las operaciones bit a bit - manipulación de registros

Un registro de un periférico de 32 bits suele alojar múltiples configuraciones independientes; cada bit o grupo de bits controla una función específica (por ejemplo, velocidad, dirección de un pin o activación de interrupciones). Por ello, escribir directamente un valor entero (como `REGISTRO_LED = 0x00000001`) suele ser peligroso, ya que sobrescribe y destruye la configuración de los 30 bits restantes.

Para modificar únicamente los bits de interés sin alterar los demás, se utiliza el patrón *Read-Modify-Write* (Lectura-Modificación-Escritura) mediante operadores bit a bit (*bitwise*):

- **Establecer bits a '1':**

```c
//ponemos el bit '0' a '1', resto se mantienen
REGISTRO_LED |= 0x00000001;
//ponemos todos los bits a '1' salvo el bit '0'
//que se mantiene
REGISTRO_LED |= 0xFFFFFFFE;
```

- **Borrar bits a '0':**

```
//ponemos el bit '0' a '0', resto se mantienen
REGISTRO_LED &= ~0x00000001;
//borramos todos los bits salvo el bit '0'
//que se mantiene
REGISTRO_LED &= 0x00000001;
```

- **Alternar estado:**

```
//cambiamos el bit '0' (de '1' a '0' o de '0' a '1')
//resto se mantienen
REGISTRO_LED ^= 0x00000001;
//cambiamos todos los bits excepto el '0'
REGISTRO_LED ^= 0xFFFFFFFE;
```

- **Leer bits individuales:**

```
//leemos el bit '0'
uint32_t valor = REGISTRO_LED & 0x00000001;
//leemos el bit '16' y lo movemos a la derecha
uint32_t valor = (REGISTRO_LED & 0x00010000) >> 16;
```

Hay que tener siempre en cuenta que los bits en los que haya que escribir, dependerán del funcionamiento del periférico mapeado en memoria. También, en ocasiones, solo leer el valor desencadena funciones del periférico. Por tanto será muy importante leer su documentación. Imaginemos un registro mapeado en memoria donde hay que modificar los bits 14 y 15 para escribir una configuración:

```
// El Pin 7 ocupa los bits [15:14] del registro GPIOA_MODER (2 bits por pin)
// Se requiere escribir el valor '01' (salida) en las posiciones 15 y 14.

// Borrar bits 15 y 14 usando una máscara invertida
GPIOA_MODER &= ~(0b11U << (7 * 2));
// 0bxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
// 0b11111111111111110011111111111111 &
// --------------------------------
// 0bxxxxxxxxxxxxxxxx00xxxxxxxxxxxxxx

// Poner el valor '01' en dichos bits
GPIOA_MODER |=  (0b01U << (7 * 2));
// 0bxxxxxxxxxxxxxxxx00xxxxxxxxxxxxxx
// 0b00000000000000000100000000000000 |
// --------------------------------
// 0bxxxxxxxxxxxxxxxx01xxxxxxxxxxxxxx
```

## 1.4. Definiciones vs variables - el espacio utilizado

En un microcontrolador, los recursos de memoria son escasos (por ejemplo, $512KB$ de Flash y $1MB$ de RAM). Es importante conocer qué recursos consume cada elemento en `C` a fin de ahorrar espacio.

- **Directivas del preprocesador (`#define`):** No consumen memoria RAM ni Flash por sí mismas. El preprocesador realiza una sustitución textual antes de la compilación.

- **Variables globales no inicializadas (`.bss`):** Se ubican exclusivamente en la RAM (pues la flash no se puede modificar directamente en tiempo de ejecución). Al arrancar el programa, el código de inicio las inicializa automáticamente a cero.
- **Variables globales inicializadas (`.data`):** Ocupan espacio en memoria Flash (donde se guarda su valor inicial de fábrica) y en memoria RAM (donde se copian durante el arranque para permitir su modificación).
- **Constantes (`const`):** Si se declaran con el calificador `const`, el compilador las ubica en la sección de solo lectura de la memoria Flash (`.rodata`), evitando ocupar memoria RAM.
- **Variables locales:** Se crean dinámicamente en la pila (*stack*) ubicada en RAM durante la ejecución de las funciones y se destruyen automáticamente al salir del ámbito.

Es importante conocer que esta distribución no es estrictamente así, ya que cambios en el script de enlazado (o *linker script*) podrán modificarla. Pero en líneas generales podemos asumir que se distribuyen de esta manera.

```
#define CONFIG_MASK  0x00FFU        // Sustitución textual, 0 bytes consumidos

const uint32_t calib = 0xF412AB00; // Memoria Flash (.rodata), 0 bytes en RAM
uint32_t contador_global;          // Memoria RAM (.bss), 4 bytes
uint32_t estado_inicial = 0xAA;    // Memoria RAM (.data) + copia en Flash

void proceso(void) {
    uint8_t buffer_local[16];      // Stack (en RAM) mientras dure la función
}
```

## 2. Estructuras y definiciones de hardware

Más allá de los accesos a memoria, para gestionar todos los elementos mapeados en memoria del procesador y periféricos, existen ciertas construcciones en C que permiten facilitar el trabajo al programador de diferentes maneras. Entre ellas, podremos evitar optimizaciones no deseadas con la palabra clave `volatile`, generar `struct` que representen periféricos, acceder a nivel de bits con los campos de bits, u optimizar llamadas a funciones `static inline`.

### 2.1. La palabra clave `volatile`

El optimizador del compilador asume un entorno de ejecución mono-hilo clásico, donde los valores almacenados en memoria solo cambian mediante instrucciones explícitas del propio código. Bajo esta premisa, si el compilador detecta lecturas repetidas a una misma dirección de memoria dentro de un bucle, optimiza el acceso cargando dicho valor en un registro interno del procesador y reutilizándolo, evitando lecturas redundantes al bus de datos.

En sistemas empotrados, esta optimización no es correcta: los registros mapeados en memoria, o las variables compartidas con Rutinas de Servicio de Interrupción (ISR) cambian de valor de forma asíncrona debido al hardware externo.

```c
// Direccion de un registro de estado del hardware (por ejemplo, UART RX Ready)
#define UART_STATUS_REG (*(uint32_t *) 0x40001000U)

void esperar_datos(void) {
    // El compilador lee el registro UNA SOLA VEZ y genera un bucle infinito
    // en ensamblador si el valor inicial era 0: "if (!reg) while(1);"
    while ((UART_STATUS_REG & 0x01U) == 0) {
        // Bucle de espera activa (Busy-wait)
    }
}
```

Al calificar una variable o puntero con `volatile`, se le indica explícitamente al compilador que no optimice sus accesos y lo lea explícitamente cada vez que el código lo requiera. Así, cada lectura o escritura escrita en `C` se traduce explíctamente en una instrucción física de lectura o escritura en la memoria.

```c
// Puntero a registro hardware: siempre calificado como volatile
#define UART_STATUS_REG (*(volatile uint32_t *) 0x40001000U)

// Variable global modificada dentro de una ISR
volatile uint8_t flag_interrupcion = 0;
```

### 2.2. Abstracción de Hardware con `struct`

Escribir macros o punteros independientes para cada uno de los decenas de registros que contiene un periférico sería, francamente, un rollo. Puesto que los registros de un periférico ocupan bloques contiguos de memoria, es habitual agruparlos dentro de un (`struct`) de `C`.

Al definir la estructura, el compilador asigna a cada miembro direcciones consecutivas de memoria desde la inicial. Bastará por tanto con configurar adecuadamente esta dirección inicial del `struct`, con la que especifiquen los manuales del procesador.

```c
#include <stdint.h>

typedef struct {
    volatile uint32_t MODER;   // Offset: 0x00 (Modo de los pines)
    volatile uint32_t OTYPER;  // Offset: 0x04 (Tipo de salida)
    volatile uint32_t OSPEEDR; // Offset: 0x08 (Velocidad de salida)
    volatile uint32_t PUPDR;   // Offset: 0x0C (Resistencias Pull-up/Pull-down)
    volatile uint32_t IDR;     // Offset: 0x10 (Registro de entrada)
    volatile uint32_t ODR;     // Offset: 0x14 (Registro de salida)
    uint32_t RESERVED[2];      // Offset: 0x18 - 0x1F (Hueco reservado en
        hardware)
    volatile uint32_t BSRR;    // Offset: 0x20 (Bit Set/Reset Register)
} GPIO_TypeDef;

// Direccion base del periférico GPIOA
#define GPIOA_BASE  (0x40020000U)

// Creación de la variable para manejar el periférico
#define GPIOA       ((GPIO_TypeDef *) GPIOA_BASE)

void ejemplo_estructura(void) {
    // Acceso directo a cualquier registro del periférico
    GPIOA->ODR   |= (1U << 7);   // Pone a '1' el Pin 7
}
```

**Alineación de memoria:** Es imprescindible respetar los huecos de direcciones no utilizadas mediante miembros de relleno (`RESERVED`), garantizando que cada miembro de la estructura coincida exactamente con la dirección del registro físico correspondiente. También hay que tener cuidado si se utilizan miembros de tipos cortos como `uint16_t` o `uint8_t`, ya que el compilador podría introducir huecos por su propia cuenta para que variables completas de tipo `uint32_t` estén alineadas a direcciones múltiplo de cuatro. Normalmente esto no es un problema, pero conviene tenerlo en cuenta.

## 2.3. Utilización de `union` y campos de bits

Aunque las operaciones bit a bit con máscaras (`&`, —) son el estándar industrial, `C` permite mapear campos de bits (*bit-fields*) mediante `struct` dentro de una `union`. Esto habilita el acceso individual a un bit o grupo de bits sin requerir desplazamientos explícitos.

```c
typedef union {
    uint32_t raw; // Acceso al registro completo de 32 bits

    struct {
        uint32_t ENABLE  : 1;  // Bit 0: Activación del periférico
        uint32_t MODE    : 2;  // Bits [2:1]: Función (00, 01, 10, 11)
        uint32_t INT_EN  : 1;  // Bit 3: Habilitar interrupción
        uint32_t RESERVED: 28; // Bits [31:4]: No utilizados
    } bits;
} CR_Register_t;

void configurar_registro(void) {
    volatile CR_Register_t *CR = (CR_Register_t *) 0x40002000U;

    // Escritura directa en campos concretos sin alterar los demás
    CR->bits.ENABLE = 1;
    CR->bits.MODE   = 0b01;

    // O bien modificación del registro completo en una sola operación
```

```
    CR->raw = 0x0000000B;
}
```

**Advertencia sobre portabilidad:** La especificación del estándar `C` deja a criterio de la implementación del compilador la ordenación de los campos de bits (de LSB a MSB o viceversa) y el empaquetado en memoria. Además, modificar un campo de bits genera internamente una secuencia *Read-Modify-Write* no atómica, lo que puede derivar en condiciones de carrera si el registro es accedido simultáneamente por una ISR.

## 2.4. Registros mayores al tamaño de palabra

Generalmente, los registros de los periféricos serán de 32 bits, con diferentes funciones para cada bit que podremos configurar, de manera sencilla, activando o desactivando dichos bits. En ocasiones, es posible encontrar periféricos con registros donde el tamaño es mayor al de palabra, y por tanto para manejar su funcionamiento, debemos movernos no sólo en los bits de un registro, sino a través de varios registros. Considera por ejemplo el siguiente código:

```
typedef struct {
    volatile uint32_t ISER[8];
    ...
    volatile uint32_t ICER[8];
    ...
} NVIC;
```

En este caso, el registro `ISER` o `ICER` tienen 256 bits de ancho (para gestionar, a través de cada uno, las diferentes fuentes de interrupción del sistema). Si queremos modificar la fuente 71, ¿cómo lo hacemos?

Si miramos los registros, veremos que en `ISER[0]` están los primeros 32 bits (0-31), en `ISER[1]` los sigientes (32-63), en `ISER[2]` los siguientes (64-95) y así sucesivamente. Por ejemplo, el bit 71 de `ISER` será, de manera equivalente, el bit 7 de `ISER[2]`. Dicho de otra manera, el resultado de **dividir** el bit entre el número de bits por sub-registro nos dará el índice de sub-registro, y el resto de dicha división será la posición a modificar dentro de ese registro. Podemos verlo en la Figura 2.

Tanto la división como el resto, al ser generalmente el divisor una potencia de dos, se pueden hacer con simples desplazamientos y máscaras bit a bit.

También cabe la posibilidad de que estos registros múltiples no tengan necesariamente un único bit por función. En estos casos, deberemos de igual manera calcular, en primer lugar, el sub-registro correspondiente mediante una división y, en un segundo lugar, el rango de bits dentro de ese sub-registro que queremos modificar mediante un módulo.

## 2.5. Palabra clave `static`

La palabra clave `static` tiene dos usos principales en `C`:

1. Hacer que una variable, dentro de una función, retenga el valor entre llamadas. Aunque no le daremos este uso durante el curso, conviene saberlo.

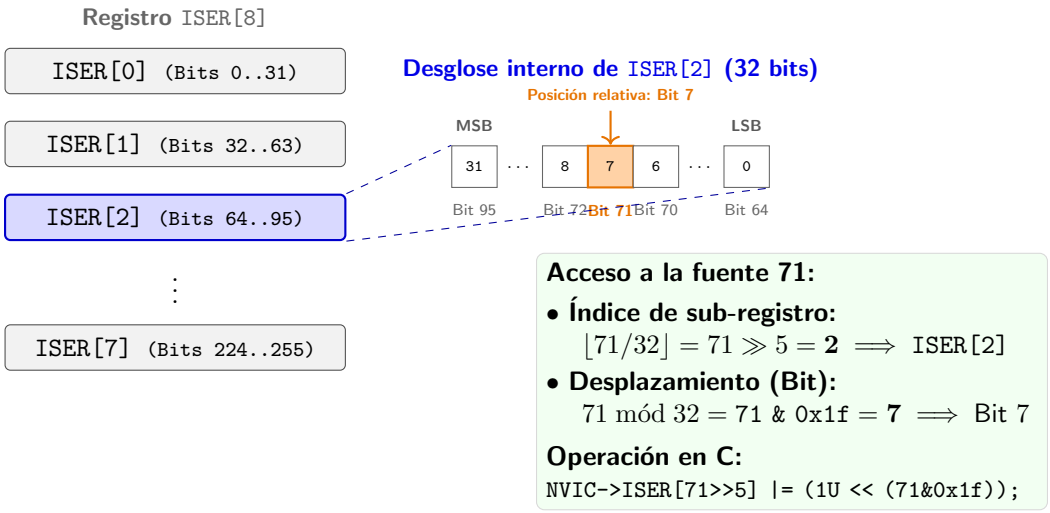



Figura 2: Esquema de direccionamiento de interrupciones en el registro `ISER[8]`.

2. Hacer que una variable o función sólo sea vista dentro de la unidad de compilación a la que pertenece. Así, esas variables o funciones no pueden ser vistas por otras unidades de compilación (normalmente archivos .c), y evitamos colisiones de nombres.

### 2.5.1. Funciones `static inline`

A la hora de realizar operaciones frecuentemente repetidas (como por ejemplo activar o desactivar un pin de un periférico de GPIO), la aproximación tradicional ha sido la de definir macros parametrizadas mediante `#define`. De esta forma, se evita la sobrecarga de llamar a una función específica, consistente en salvar registros en la pila, además de realizar el salto y retorno.

```
#define TOGGLE_PIN_MACRO(port, pin)  ((port)->ODR ^= (1U << (pin)))

// Evaluacion con incremento: TOGGLE_PIN_MACRO(GPIOA, i)
// expande a: (GPIOA)->ODR ^= (1U << (i))
```

Las macros, sin embargo, presentan algunos inconvenientes, especialmente cuando empieza a haber muchas: La trazabilidad se hace complicada, tanto en el desarrollo como en la depuración. Adicionalmente, los errores de sintaxis se hacen más comunes debido a las reglas de sustitución del preprocesador.

Como alternativa, disponemos de las funciones `static inline`. La palabra clave `inline` hace que el compilador reemplace la llamada a la función directamente por su código fuente el el punto de llamada. Por su parte, `static`, como ya decíamos, hace que la función no sea vista fuera de la unidad de compilación, así que aunque la incluyamos desde numerosos archivos `.c`, evitaremos errores de duplicidad al compilar.

```
#include <stdint.h>

static inline void gpio_toggle_pin(GPIO_TypeDef *port, uint8_t pin) {
    // Comprobación de tipos en compilación y sin efectos secundarios
    port->ODR ^= (1U << pin);
}
```

Así, se consigue la rapidez de las macros, añadiendo las comprobaciones extra del compilador

y funciones de depuración estándar.

# 3. El enlazado de programas e inicialización

Para convertir varios archivos fuente compilados en un único ejecutable funcional, el enlazador (*linker*) asigna direcciones físicas de memoria a cada instrucción y variable. Aunque para las instrucciones nos importa un poco menos, es importante asegurar que ciertos elementos de nuestro programa acaban en direcciones de memoria concretas. Por ejemplo, el código de arranque o ciertas funciones de interrupción (disparados ambos automáticamente) tendrán que estar en ubicaciones concretas.

## 3.1. El *linker script*

El *linker script* (archivo con extensión `.ld`) es el documento de configuración que indica al enlazador cómo organizar la memoria del sistema. Cumple tres funciones principales: define las regiones de memoria física (utilización de Flash y RAM), ubica las diferentes secciones de código (variables/funciones) en ellas, y además permite exportar símbolos de dirección para que el propio código sepa dónde acaba ubicado.

Un mapa de memoria típico de ARM Cortex-M divide las secciones del programa en:

- `.isr_vector`**:** Tabla de punteros con las llamadas a funciones de inicio e interrupciones.
- `.text`**:** Código ejecutable e instrucciones del programa (almacenado en Flash).
- `.rodata`**:** Constantes y cadenas de texto de solo lectura (almacenado en Flash).
- `.data`**:** Variables globales e estáticas *inicializadas* con un valor distinto de cero.
- `.bss`**:** Variables globales e estáticas *no inicializadas* (o inicializadas a cero).

```
MEMORY
{
    RAM (xrw)    : ORIGIN = 0x20000000, LENGTH = 256K
    FLASH (rx)   : ORIGIN = 0x08000000, LENGTH = 512K
}

SECTIONS
{
    /* Tabla de vectores al inicio de la Flash */
    .isr_vector : {
        KEEP(*(.isr_vector))
    } >FLASH

    /* Código */
    .text : {
        *(.text*)
        *(.rodata*)
        _etext = .; //símbolo exportado al final del código
    } >FLASH

    /* .rodata (constantes, strings, etc.) */
    .rodata :
    {
        *(.rodata)
        *(.rodata*)
    } >FLASH
```

```
27
28      /* Datos inicializados, dirección de comienzo en FLASH */
29      _sidata = LOADADDR(.data);
30
31      .data :
32      {
33          _sdata = .;         /* Inicio de datos en RAM */
34          *(.data)            /* .data sections */
35          *(.data*)           /* .data* sections */
36          _edata = .;         /* Fin de datos en RAM*/
37
38      } >RAM AT> FLASH /* Se guarda en RAM en ejecución, pero está en
39      FLASH en compilación */
40
41      /* Datos sin inicializar van a RAM directamente */
42      .bss : {
43          _sbss = .;
44          *(.bss*)
45          _ebss = .;
46      } > RAM
47
48      /* Definicion del tope de la pila al final de la RAM */
49      _estack = ORIGIN(RAM) + LENGTH(RAM);
50  }
```

Es importante diferenciar la **LMA** (*Load Memory Address*) frente a **VMA** (*Virtual Memory Address*). La sección `.data` reside físicamente en Flash (**LMA**) para no perder sus valores al apagar el chip, pero el procesador debe ejecutarla y modificarla desde la RAM (**VMA**). Este tipo de inicialización la hace el código de inicio.

### 3.2. Código de inicio y salto a `main`

Cuando el microcontrolador recibe alimentación o se resetea, el procesador no ejecuta directamente la función `main()`. En su lugar, el hardware lee la primera dirección de memoria y salta automáticamente al `Reset_Handler`, definido en el archivo de ensamblador de arranque (`startup.s` o similar).

El `Reset_Handler` realiza la inicialización básica del sistema en el siguiente orden estricto:

1. **Inicialización del Stack Pointer (SP):** Carga la dirección del puntero de pila principal con el símbolo `_estack`.
2. **Llamada a `SystemInit`:** Configura los relojes principales (*clocks*) y la memoria Flash.
3. **Copia de la sección `.data`:** Lee el bloque de inicializadores desde la Flash (`_sidata`) y los escribe en la RAM (desde `_sdata` hasta `_edata`).
4. **Limpieza de la sección `.bss`:** Rellena con ceros la región de RAM comprendida entre `_sbss` y `_ebss`.
5. **Llamada a constructores globales (`__libc_init_array`):** Prepara librerías de C/C++.
6. **Salto a `main()`:** Salta a la rutina principal de la aplicación. Si esta finaliza, entra en un bucle infinito de seguridad (`LoopForever`).

```
Reset_Handler:
    ldr   r0, =_estack
    mov   sp, r0          /* set stack pointer */
    /* Call the clock system initialization function.*/
    bl  SystemInit

    /* Copy the data segment initializers from flash to SRAM */
    ldr r0, =_sdata
    ldr r1, =_edata
    ldr r2, =_sidata
    movs r3, #0
    b LoopCopyDataInit

/***** Sección de copia de datos inicializados *****/
CopyDataInit:
    ldr r4, [r2, r3]
    str r4, [r0, r3]
    adds r3, r3, #4

LoopCopyDataInit:
    adds r4, r0, r3
    cmp r4, r1
    bcc CopyDataInit

/***** Sección de inicialización con ceros de .bss *****/
    ldr r2, =_sbss
    ldr r4, =_ebss
    movs r3, #0
    b LoopFillZerobss

FillZerobss:
    str  r3, [r2]
    adds r2, r2, #4

LoopFillZerobss:
    cmp r2, r4
    bcc FillZerobss

/***** Inicialización de librerías *****/
    bl __libc_init_array

/***** Salto al programa principal *****/
    bl main

LoopForever:
    b LoopForever
```

## 3.3. La tabla de vectores

La arquitectura ARM Cortex-M requiere que las direcciones de las funciones de atención a excepciones e interrupciones estén organizadas en una posición fija de memoria, conocida como **Tabla de Vectores**. Por defecto, se ubica al principio de la memoria Flash (`0x08000000`).

Cada entrada de la tabla es una palabra de 32 bits que contiene la dirección de memoria de la rutina correspondiente:

- **Offset `0x00`:** Puntero inicial de la pila (`_estack`).

- **Offset `0x04`:** Dirección del `Reset_Handler`.
- **Offset `0x08` a `0x3C`:** Excepciones del núcleo Cortex (NMI, HardFault, SysTick, etc.).
- **Offset `0x40` en adelante:** Interrupciones de periféricos propios del fabricante (USART, TIM, DMA, etc.).

```
.section .isr_vector,"a",%progbits
.type g_pfnVectors, %object

g_pfnVectors:
.word _estack                /* 0x00: Pila inicial */
.word Reset_Handler          /* 0x04: Reset Handler */
.word NMI_Handler            /* 0x08: NMI Handler */
.word HardFault_Handler      /* 0x0C: HardFault Handler */
/* ... Excepciones del nucleo ... */
.word SysTick_Handler        /* 0x3C: Timer del sistema */

/* Interrupciones de perifericos externos */
.word WWDG_IRQHandler
.word USART1_IRQHandler
.word EXTI0_IRQHandler
```

Podemos observar que esta sección (`.isr_vector`) es la que colocaba anteriormente el *linker script* al principio exacto de la **FLASH**, para así ubicar todos estos símbolos en las posiciones adecuadas.

### 3.3.1. El modificador *weak*

En el archivo de arranque, todas las interrupciones se enlazan por defecto a una rutina genérica llamada `Default_Handler` (que ejecuta un bucle infinito) mediante el atributo de ensamblador `.weak` (o `__attribute__((weak))` en C).

El modificador *weak* (débil) indica al enlazador que use esa definición por defecto únicamente si el usuario no ha definido una función con el mismo nombre en su código C.

```
.weak USART1_IRQHandler
.thumb_set USART1_IRQHandler, Default_Handler
```

Si el desarrollador escribe en su archivo `main.c`:

```
void USART1_IRQHandler(void) {
    // Codigo propio para procesar la recepcion de datos
}
```

El enlazador sustituye automáticamente la dirección del `Default_Handler` por la dirección de esta nueva función dentro de la tabla de vectores, sin necesidad de modificar el archivo de ensamblador original.

# 4. Interrupciones, sincronización y *callbacks*

Cuando salta una interrupción (cuya dirección está almacenada en la tabla de vectores), el hardware del procesador suspende la ejecución del hilo principal (`main`), guarda automáticamente el contexto (registros `R0-R3`, `R12`, `LR`, `PC`, `xPSR`) en la pila y salta a la ISR asignada en la tabla de vectores.

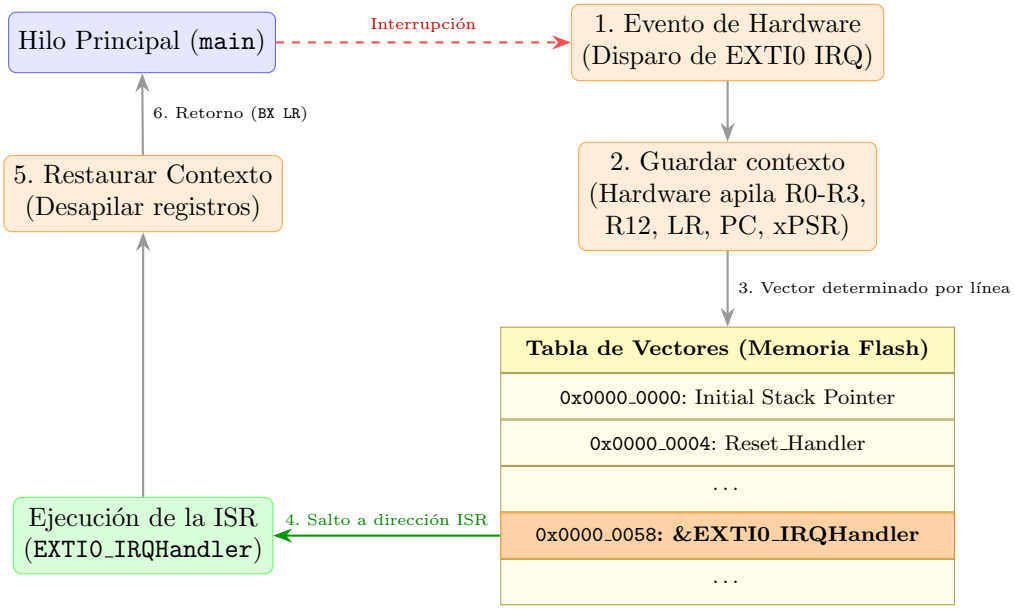



Figura 3: Mecanismo de atención a interrupciones y lectura de puntero en la Tabla de Vectores.

Al compartir variables entre el hilo principal y una rutina de interrupción, surgen problemas de inconsistencia de datos y condiciones de carrera:

- **Uso obligatorio de `volatile`:** Evita que el compilador almacene en registros de la CPU el valor de una variable que puede ser alterada por la ISR en cualquier momento.
- **Operaciones no atómicas:** Si el hilo principal modifica una variable de 32 bits mientras ocurre una interrupción a mitad del proceso de escritura, la ISR leerá datos corruptos. Normalmente, las variables de 32 bits ya se gestionan de manera atómica automáticamente en ARM, pero es mejor protegerlas por si las moscas en caso de poder existir condiciones de carrera.

```c
#include <stdint.h>

volatile uint32_t contador_eventos = 0;

void TIM2_IRQHandler(void) {
    // esta función se ejecuta automáticamente cuando hay interrupción
    contador_eventos++;
}

uint32_t leer_contador_seguro(void) {
    uint32_t copia;

    // Desactivar interrupciones antes de leer
    __asm volatile ("cpsid i" : : : "memory");
    copia = contador_eventos; // Lectura protegida
```

```
    // Reactiva interrupciones tras leer
    __asm volatile ("cpsie i" : : : "memory");

    return copia;
}
```

## 4.1. Barreras de memoria

El código `C` mantiene un orden lógico. Sin embargo, las optimizaciones del compilador, y el reordenamiento dinámico de instrucciones que sucede a nivel arquitectural, pueden cambiar dicho orden, a fin de mejorar el rendimiento. Esto siempre se hace de tal manera que el resultado final es el mismo, siempre y cuando no haya modificaciones externas de los valores implicados. ¿Y qué pasa con un sistema empotrado con periféricos mapeados en memoria? Pues que puede haber modificaciones externas.

En ciertos momentos, querremos obligar al procesador a completar las operaciones de memoria antes de continuar. Para ello, la arquitectura ARM Cortex-M provee tres instrucciones de barrera de memoria:

- `__DMB()` **(Data Memory Barrier):** Garantiza que todas las accesos a memoria (lecturas y escrituras) previos a la barrera se hayan completado antes de que se inicie cualquier acceso a memoria posterior. No frena la ejecución de instrucciones que no accedan a memoria.

```
__attribute__((always_inline)) static inline void __DSB(void) {
    __asm volatile ("dmb 0xF":::"memory");
}
```

- `__DSB()` **(Data Synchronization Barrier):** Detiene la ejecución de **todas** las instrucciones hasta que las operaciones de acceso a memoria pendientes hayan finalizado.

```
__attribute__((always_inline)) static inline void __DMB(void) {
    __asm volatile ("dsb 0xF":::"memory");
}
```

- `__ISB()` **(Instruction Synchronization Barrier):** Completa todas las instrucciones pendientes del procesador y fuerza a volver a cargar las instrucciones desde memoria o caché. Se utiliza tras modificar la configuración del sistema (como el mapa de memoria o la MPU) para evitar mantener valores antiguos.

```
__attribute__((always_inline)) static inline void __ISB(void) {
    __asm volatile ("isb 0xF":::"memory");
}
```

Generalmente, los manuales de uso nos indicarán cuándo es necesario utilizar una de estas barreras al interactuar con los diferentes periféricos y elementos del procesador.

Un caso de uso de `__DSB()` en microcontroladores ocurre al limpiar bits de interrupción en una ISR. Debido al tiempo que tarda el bus en procesar la escritura en el periférico (potencialmente miles de ciclos si el reloj del periférico es muy lento), la CPU podría salir de la ISR antes de que la bandera se limpie físicamente en el registro, re-disparando la misma interrupción.

```c
void EXTI0_IRQHandler(void) {
    // ... gestión de ISR

    // ... limpieza de bits
    EXTI->PR = EXTI_PR_PR0;
    // ... garantizamos que se haya escrito antes de salir
    __DSB();
}
```

## 4.2. Punteros a función y Callbacks

Para mantener un diseño de software modular, la capa que interactúa directamente con el hardware (*drivers*) no debe contener lógica de aplicación. El mecanismo estándar para notificar eventos desde dichos *drivers* (o una ISR) hacia la aplicación es la utilización de **punteros a función**. Esta técnica se conoce comúnmente como *callbacks*, ya que la función *hardware* hará una llamada (*callback*) a nuestra función con lógica.

Un puntero a función almacena la dirección de memoria donde está situada, permitiendo su invocación dinámica en tiempo de ejecución. Podemos ver un esquema en la Figura 4.

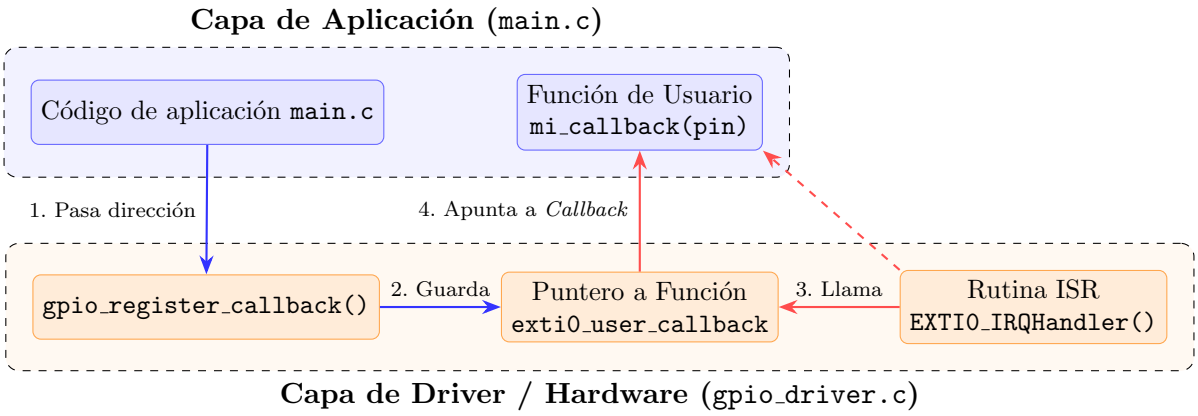



Figura 4: Mecanismo de Registro y Ejecución de un Call-back.

Hay que tener en cuenta que los *callback* se ejecutan desde la interrupción, por tanto el procesador está en modo de atención de ISRs. Estos callbacks no deben incluir procesamiento pesado, y únicamente marcar los valores necesarios o tareas pendientes para que el hilo principal, al volver, los gestione. De lo contrario, estaríamos retrasando cualquier otra llamada a interrupción durante la duración de ejecución del *callback*.

```c
#include <stddef.h>
#include <stdint.h>

// Definición del tipo de puntero a función
// Será una función que no devuelve nada (void)
// Y que recibe un argumento tipo uint16_t
typedef void (*gpio_callback_t)(uint16_t pin);

// Variable del driver para almacenar el callback
static volatile gpio_callback_t exti0_user_callback = NULL;

// Función de la API pública para registrar el callback
// desde el código de aplicación (p.ej: main.c)
```

```
14 void gpio_register_callback(gpio_callback_t callback) {
15     exti0_user_callback = callback;
16 }
17
18 // Manejador de la interrupción del hardware
19 void EXTI0_IRQHandler(void) {
20     // ... procesamiento de ISR
21     uint16_t pin = ...;
22
23     // limpieza de bits de interrupción
24     EXTI->PR = EXTI_PR_PR0;
25     __DSB();
26
27     // Llamada al callback
28     if (exti0_user_callback != NULL) {
29         exti0_user_callback(pin);
30         // Ejecuta la función de la capa de aplicación
31         // avisando del estado (en este caso el pin)
32     }
33 }
```

# 5. Los periféricos del procesador

Un procesador, o la idea de procesador que podemos tener en general (máquina que ejecuta instrucciones secuencialmente) es sólo una pequeña parte de toda la historia. La parte ejecutora de instrucciones es el núcleo o *core*. Con esto, el procesador sólo sería capaz de procesar instrucciones... y poco más.

Para delegar funciones, y permitir interconectarlo con el exterior, un procesador se rodea de periféricos. Estos periféricos se pueden configurar para realizar, de manera autónoma, funciones que de otra manera tendrían ocupado al procesador. Algunos de los más comunes son temporizadores, manejadores de protocolos de intercomunicación, aceleradores hardware para funciones complejas (seguridad, criptografía...), motores de acceso a memoria directo (DMAs), etc.

Todos estos periféricos se manejan desde direcciones de memoria, donde se mapean todos los registros internos. En nuestro caso concreto, y como ejemplo, el procesador **STM32F723IE** cuenta con un bus principal de memoria de tipo AXI (*Advanced Extensible Interface*) para enviar y recibir datos del espacio de memoria. Numerosos buses secundarios de tipo AHB (*Advanced High-Performance Bus* se conectan a través de una matriz de interconexión ( `rm0431 2` ), que a su vez sirven diferentes subconjuntos de dispositivos como se muestra en la Figura 5.

Todas estas interconexiones entre buses se identifican, a nivel de hardware, mediante direcciones. En cada punto, y según la dirección de memoria solicitada, se establecerá una conexión entre el origen y destino de la transferencia de datos. Esto es automático desde el punto de vista del procesador o periféricos, que únicamente tendrán que colocar sus datos (o peticiones) y esperar a la confirmación de transacción exitosa.

## 5.1. El bus AXI

El bus *AXI* (*Advanced Extensible Interface*) es el medio que tiene el procesador de enviar sus peticiones al exterior. Perteneciente a la especificación AMBA (*Advanced Microcontroller Bus Architecture*) de ARM, es una interfaz de bus con gran ancho de banda y frecuencia, y baja latencia.

AXI se basa en un esquema de cinco canales independientes para la transferencia de datos y direcciones:

- **Canal de dirección de lectura (*Read Address*):** Especifica la dirección de memoria para lectura y señales de control.
- **Canal de datos de lectura (*Read Data*):** Transporta los datos leídos desde el esclavo hacia el maestro, junto con la respuesta de estado (*OKAY*, *ERROR*, etc.).
- **Canal de dirección de escritura (*Write Address*):** Especifica la dirección de memoria para escritura y señales de control.
- **Canal de datos de escritura (*Write Data*):** Transporta los datos a escribir.
- **Canal de respuesta de escritura (*Write Response*):** Permite notificar al maestro del resultado final de la operación de escritura.

El dividir el bus en canales de subida y bajada permite lecturas y escrituras simultáneas. Adicionalmente, las señales internas permiten transferencias en ráfaga o incluso fuera de orden.

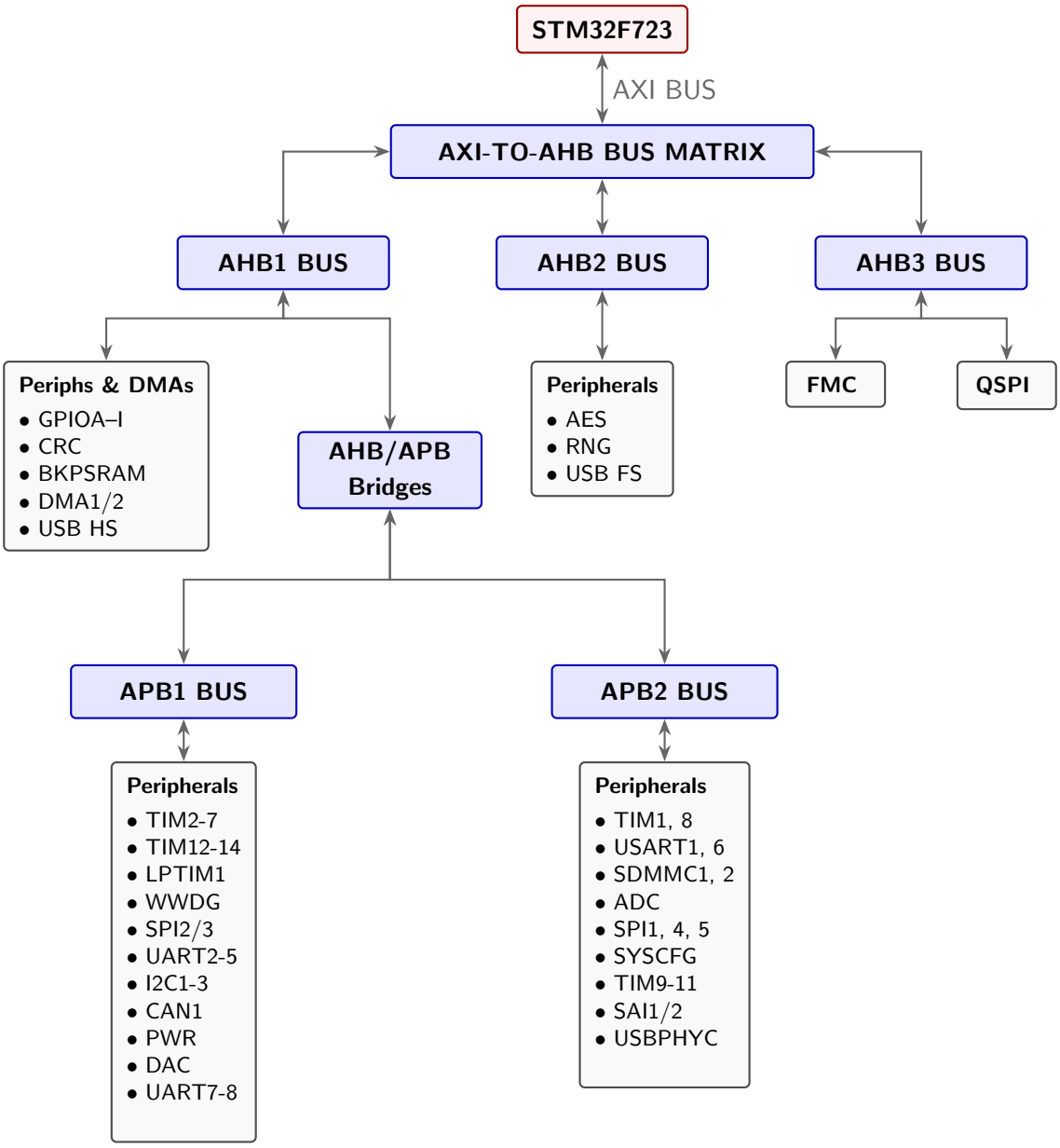



Figura 5: Interconexión del procesador y dispositivos

### 5.1.1. *Handshake* o sincronización

Todas las transferencias en cada uno de los 5 canales de AXI utilizan un mecanismo uniforme de sincronización denominado *handshake* bidireccional mediante dos señales principales:

- `VALID`: Generada por el emisor (origen) para indicar que las direcciones, datos o señales de control presentes en el canal son válidos.
- `READY`: Generada por el receptor (destino) para indicar que está listo para aceptar la información.

La transferencia de datos ocurre exclusivamente en el flanco de reloj ascendente en el que **ambas** señales (`VALID` y `READY`) se encuentran activas en nivel alto (1), tal como muestra la Figura 6.

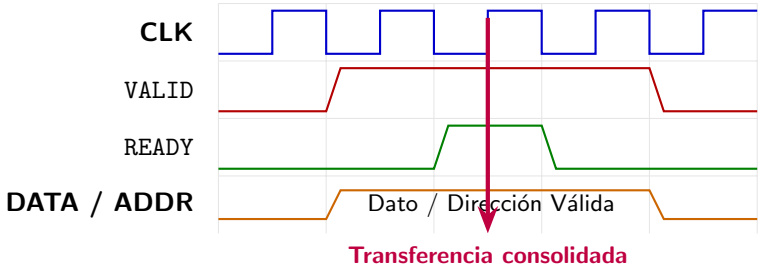



Figura 6: Sincronización por *handshake* en un canal AXI. La transferencia se realiza únicamente cuando `VALID` y `READY` coinciden en nivel alto durante el flanco del reloj.

## 5.2. El bus AHB y APB

El bus *AHB* interconecta el procesador con las memorias internas (SRAM, Flash) y los periféricos (como los controladores DMA, Ethernet o USB). Aunque se podría utilizar AXI, AHB es más sencillo y evita una compleja lógica de control en cada uno de los periféricos, mientras que mantiene un rendimiento elevado.

Su funcionamiento es mucho más sencillo, con dos fases (envío de dirección y envío de datos) que se suceden en el tiempo. Para mejorar el rendimiento, estas fases se pueden solapar formando un pipeline.

Los periféricos sencillos o de baja velocidad no se conectan directamente al bus de altas prestaciones (AHB) debido a sus menores requerimientos de velocidad y a la necesidad de no recargar el bus principal. Para ello se emplean buses de periferia más simples, como el bus *APB* (*Advanced Peripheral Bus*), y elementos de adaptación denominados **puentes** (*bridges*).

## 5.3. El puente entre bus y registro

Un puente (*Bridge*) actúa como un esclavo dentro del bus de alta velocidad (AHB) y como el único maestro en el bus periférico (APB). Su función principal es traducir las transacciones de alta velocidad de AHB en lecturas y escrituras de 2 ciclos de reloj (dirección y dato) en APB.

Cuando se realiza una lectura o escritura en una dirección de memoria correspondiente a un periférico, se sigue el siguiente flujo:

1. **Decodificación en bus/puente:** El decodificador examina los bits superiores de la dirección del bus hasta identificar el periférico correspondiente.
2. **Selección de Periférico:** El puente activa la señal de selección individual correspondiente al periférico deseado (`PSELx`).
3. ***Offset* de Registro:** Los bits inferiores de la dirección seleccionan el registro interno dentro del periférico.
4. **Escritura / Lectura:** El periférico lee o escribe en el registro indicado los datos indicados.

## 5.4. La interconexión con diferentes memorias

La interconexión con las memorias se hace de manera similar a como se hace con el resto de los periféricos. Al fin y al cabo, recibirán una dirección y dato, además de una operación (escritura/lectura) y con tan solo esa información podrán operar. Sin embargo, además de los buses habituales, suelen existir otros buses secundarios que ofrecen accesos directos.

### 5.4.1. Memorias acopladas mediante ITCM y DTCM

Para evitar saturar la matriz de buses cuando el núcleo ejecuta instrucciones a frecuencias muy elevadas, el procesador suele integrar interfaces de memorias con cero estados de espera (*Zero Wait State*).

- **ITCM (*Instruction Tightly-Coupled Memory*):** Bus y memoria dedicados exclusivamente a la búsqueda y ejecución de código.
- **DTCM (*Data Tightly-Coupled Memory*):** Bus y memoria dedicados a datos críticos de tiempo real.

### 5.4.2. Controladores de memoria externa (FMC, QSPI, Flash...)

Si los requisitos del sistema piden más memoria que la alcanzable con las anteriores interfaces, se utilizan periféricos, ya a través de los buses AHB, que interconectan con memorias externas de amayores tamaños. Por ejemplo:

- **FMC (*Flexible Memory Controller*):** Permite conectar memorias externas paralelas como SDRAM, SRAM o NOR Flash.
- **QSPI (*Quad-SPI Interface*):** Interfaz serie de alta velocidad para memorias Flash externas.

## 5.5. Ejemplos de periféricos

Los periféricos integrados en el microcontrolador abstraen al procesador de tareas de bajo nivel. Por ejemplo, para interconectar mediante dispositivos externos con protocolos como USB rm0431 32,33, UART rm0431 27, SPI rm0431 28... en lugar de tener al procesador ocupado, tenemos periféricos que autogestionan la conexión y avisan al procesador sólo cuando hay una decisión que tomar.

También la gestión de entrada/salida mediante pines de propósito general es controlada mediante los periféricos GPIO rm0431 6, por ejemplo para encender LEDs o leer el estado de botones.

O todo lo referente a temporización del sistema, mediante periféricos como los temporizadores TIM rm0431 21 o el reloj de tiempo real RTC rm0431 25.

## 5.6. Activación de periféricos

Un aspecto muy a tener en cuenta en microcontroladores modernos es la gestión de energía. Por defecto, todos los periféricos del microcontrolador están desactivados tras el reseteo (sus señales de reloj están desconectadas).

Para conectarlas y dar servicio a los periféricos, se deben activar uno a uno mediante señales específicas. Normalmente, un periférico es el responsable de esta activación, y en concreto en el **STM32F723IE** es el RCC rm0431 5.

El periférico RCC rm0431 5 gestiona los relojes del sistema (origen, frecuencia, forma...) y la habilitación individual del reloj de cada periférico.

Antes de poder leer o escribir en cualquier registro de un periférico, habrá que haber activado previamente su señal de reloj en el registro RCC correspondiente.

# 6. Temporización

En el centro de todo procesador está el reloj. Es quien, en cada ciclo, dicta cuándo los datos tienen que moverse de un lugar a otro, haciendo que se actualicen los registros con nuevos valores, cuya propagación prepara los resultados para el siguiente ciclo... y así sucesivamente. En los procesadores de hoy en día, no existe un solo reloj, sino que multitud de diferentes fuentes proveen de señales de reloj distintas a todos sus componentes.

Aparte de llevar la actualización secuencial de los datos, el reloj también sirve como base para contar el tiempo que sucede entre diferentes eventos del sistema. Así, se puede controlar el tiempo de ejecución de tareas en un SSOO, las señales PWM que controlan la intensidad de LEDs o velocidad de motores, medir frecuencias y velocidades externas, conversiones digital-analógica, despertar al procesador tras un tiempo determinado... Para ello, contamos con diferentes periféricos que nos facilitarán la gestión de todas estas tareas.

¿Pero primero, cómo empieza a generarse la señal de reloj?

## 6.1. Osciladores

La base de cualquier tipo de temporización es el oscilador, que proporciona una señal cuadrada para utilizar como reloj del sistema. Principalmente existen dos tipos: integrados en la CPU, de menor precisión pero disponibles *out of the box*, y externos, que suelen ser más precisos, pero requieren de espacio adicional en la PCB.

Los osciladores integrados internos funcionan alimentando un pequeño condensador con una corriente continua. Esto hace que se vaya llenando y su voltaje se incremente poco a poco. Un comparador de voltaje detecta cuando este voltaje cruza un umbral de referencia, descargando el condensador inmediatamente mediante un transistor. Este proceso se repite sucesivamente, alternando el valor de un flip-flop y generando así la onda cuadrada necesaria para el reloj. Debido a las diferencias de fabricación, la velocidad de recarga y el umbral de referencia no son necesariamente fijos, por lo que se guardan valores de calibración en el procesador para que todas las unidades funcionen, aproximadamente, a la misma velocidad.

Aún así, la resistencia, capacitancia y voltaje son dependientes de la temperatura, por lo que estas fuentes de reloj pueden variar en frecuencia según parámetros ambientales, por lo que no se utilizan en aplicaciones de precisión.

Los osciladores externos dan, desde el exterior, una onda de reloj al procesador. Los más comunes se basan en un cristal de cuarzo que, sometido a un campo eléctrico, vibra a una frecuencia determinada dada por su forma. Controlando los procesos de fabricación, se puede controlar esa frecuencia, que en cualquier caso será fija una vez fabricado.

Los osciladores integrados suelen tener una tolerancia de $\pm 1\,\%$, mientras que los externos de cristal de cuarzo oscilan entre tolerancias del $\pm 0{,}005\,\%$ hasta $\pm 0{,}00005\,\%$. Esto quiere decir que, mientras un oscilador interno puede desviarse hasta 15 minutos al día, un oscilador de cristal de alta precisión solo se desvía, como mucho, 1-5 segundos por año. Los modelos más precisos (para aplicaciones de temporización crítica) son capaces de precisiones de fracciones de milisegundo al año. Para precisiones aún mayores (como el GPS) se utilizan relojes atómicos basados en la descomposición de ciertos elementos radiactivos.

Adicionalmente, se suele contar con osciladores de baja (low) y alta (high) frecuencia, para

uso en diferentes elementos del procesador (el núcleo irá a alta frecuencia, mientras que funciones de bajo consumo irán a baja frecuencia). El oscilador por defecto del **STM32F723IE**es el HSI (*high speed internal oscillator*) a 16MHz de frecuencia.

## 6.2. Árbol de reloj

Normalmente no se dispone de un solo oscilador (ya sea interno o externo) sino que se tienen varios para uso en diferentes elementos del procesador. Por ejemplo, el núcleo interesa que vaya a alta frecuencia, mientras que funciones de bajo consumo vayan a baja frecuencia. Pero si sólo tuviéramos un oscilador rápido, lo segundo sería imposible. Así, en numerosas ocasiones se dispone de hasta 4 osciladores:

- HSI: *High speed internal:* De alta velocidad sintetizado por el propio silicio
- LSI: *Low speed internal:* De baja velocidad sintetizado por el propio silicio
- HSE: *High speed external:* De alta velocidad proveniente de un cristal de cuarzo.
- LSE: *Low speed external:* De baja velocidad proveniente de un cristal de cuarzo.

Los de alta velocidad suelen llegar a velocidades de MHz, mientras que los de baja velocidad se mantienen en los KHz.

El árbol de reloj es la estructura que conecta cada uno de los relojes a los componentes que queramos. Adicionalmente, dispone de multiplicadores y divisores de frecuencia para poder dar diferentes velocidades de reloj a los periféricos (especialmente interesante para protocolos con una frecuencia base fija estandarizada). Se muestra un ejemplo de árbol de reloj en la Figura 7

## 6.3. Ticker del sistema

El *ticker* del sistema es un temporizador simple integrado directamente en el núcleo de la mayoría de procesadores modernos. A diferencia de los periféricos de temporización estándar, su propósito no es controlar hardware externo, sino proporcionar una base deL tiempo transcurrido desde el inicio, para la gestión interna del software.

Funciona a partir de un registro contador que se decrementa con cada pulso del reloj del sistema. Al llegar a cero, se recarga automáticamente con un valor preconfigurado (*auto-reload*) y genera una interrupción del sistema. Si configuramos esta recarga para que esta interrupción suceda a un intervalo regular (cada 1 ms es lo más habitual), se obtiene el *tick* sobre el cual se construyen varias funcionalidades:

- **Marcas de tiempo:** Permite implementar contadores de milisegundos desde el arranque (como la función `millis()`) para medir tiempos transcurridos o gestionar *timeouts* sin congelar la ejecución del hilo principal del procesador. Esto se hace manteniendo una variable que se incrementa en cada *tick*, la cual se comprueba para ver si toca realizar alguna función.
- **Planificación en RTOS:** En sistemas operativos en tiempo real, la interrupción del *ticker* es aprovechada por el *scheduler* para realizar cambios de contexto entre tareas, gestionar tiempos de espera (*delays*) y controlar recursos.

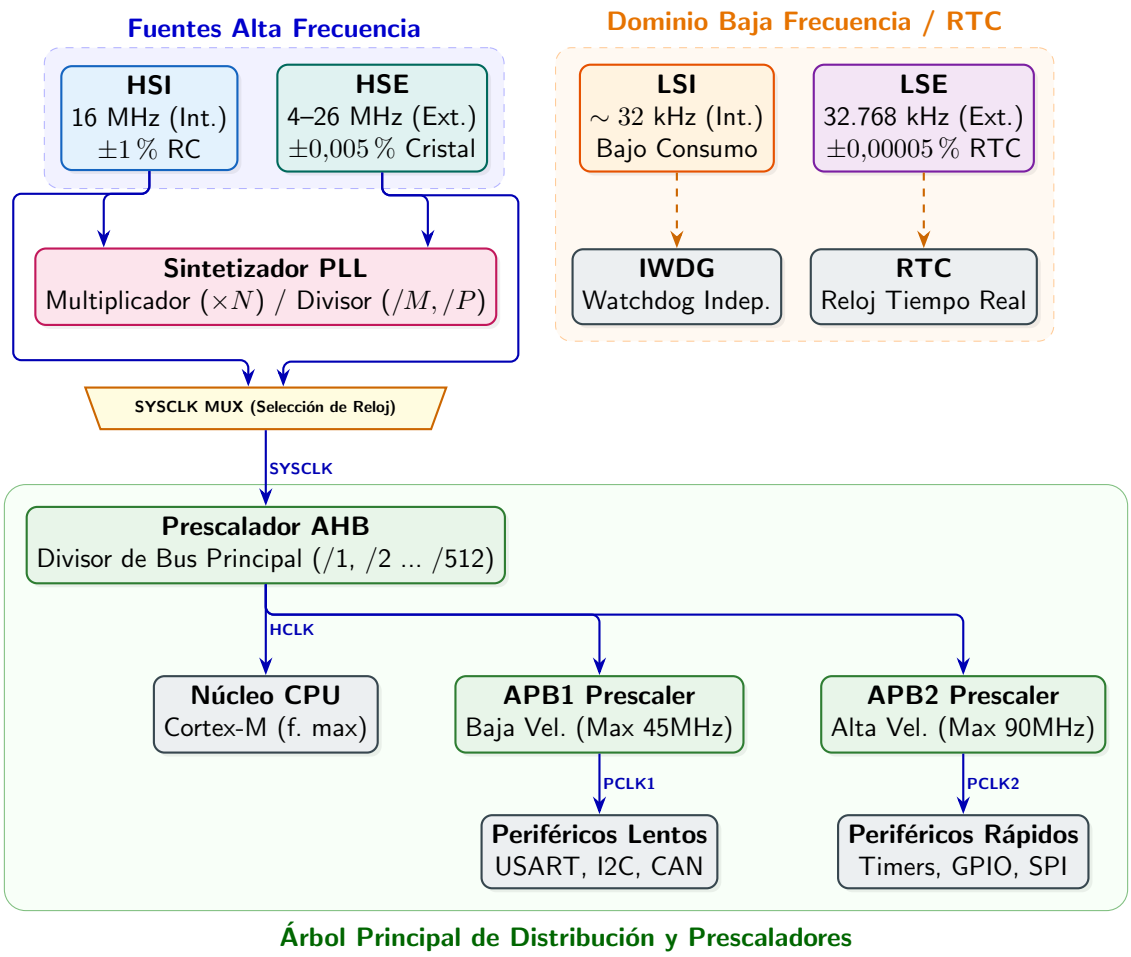



Figura 7: Ejemplo de árbol de reloj con diferentes fuentes y destinos

Al estar acoplado al núcleo, y generalmente tratarse de una interrupción de alta prioridad, se garantiza que esta gestión temporal no se vea afectada por otras configuraciones.

## 6.4. Temporizadores

Mientras que el *ticker* atiende necesidades del núcleo, los **temporizadores de propósito general** (*Timers*) son periféricos independientes orientados a la interacción con el entorno externo y el hardware. En esencia, los temporizadores son contadores que se incrementan o decrementan según una fuente de pulsos de reloj, y son capaces de disparar interrupciones al cumplir ciertas condiciones sus valores. Casi cualquier temporizador genérico consta de tres componentes principales:

1. **Prescalador (PSC):** Es un divisor de frecuencia previo al contador. Si el reloj de entrada es de 84 MHz, un prescalador ajustado a 84 hará que el contador aumente a una frecuencia de exactamente 1 MHz (cada 1 $\mu$s).
2. **Contador (CNT):** Registro central que almacena el valor numérico actual del recuento.
3. **Registro de auto-recarga (ARR / Period):** Establece el límite máximo de cuenta. Cuando $CNT = ARR$, el contador se reinicia a cero.

La frecuencia final de reinicios (o eventos) ($f_{\text{evento}}$) viene dada por la ecuación:

$$f_{\text{evento}} = \frac{f_{\text{CLK}}}{(PSC + 1) \cdot (ARR + 1)}$$

Gracias a la inclusión de canales de captura y comparación (*Capture/Compare*), los temporizadores pueden operar de diversas formas autónomas:

- **Modo Comparación y PWM (*Pulse Width Modulation*):** El valor actual del contador se compara continuamente con un valor almacenado en un registro de comparación ($CCR$). Cuando $CNT < CCR$, una pin de salida digital se mantiene en estado alto, y cuando $CNT \geq CCR$, pasa a estado bajo. Modificando el registro $CCR$ se controla el ancho del pulso (ciclo de trabajo) para regular la potencia de motores, la intensidad de LEDs o sintetizar señales analógicas.
- **Captura de entrada (*Input Capture*):** Ante un flanco de subida o bajada en un pin externo (interrupción), el valor del contador en ese preciso instante se copia a un registro de captura. Esto permite medir con precisión de microsegundos la duración de un pulso externo, medir frecuencias o interpretar sensores ultrasónicos y receptores de radiofrecuencia.
- **Temporizadores de supervisión (*Watchdog*):** Son temporizadores de seguridad cuya función es reiniciar el sistema si el software se bloquea en un bucle infinito. La aplicación debe .ªlimentar"periódicamente al *watchdog*; si el contador alcanza el límite, se asume un fallo catastrófico y se fuerza un reinicio.
- **Temporizadores estándar:** Simplemente esperan el tiempo configurado en el temporizador para generar una interrupción. Por ejemplo son utilizados para eliminar rebotes de teclas, generar animaciones en pantallas...

## 6.5. RTC (*Real-Time Clock*)

El Reloj en Tiempo Real o **RTC** mantiene la hora de calendario (segundos, minutos, horas, día, mes y año) de forma ininterrumpida. A diferencia de un temporizador convencional diseñado para medir lapsos breves en escala de microsegundos, el RTC es capaz de medir tiempo a largo plazo y con un consumo de energía extremadamente bajo. Esto es útil para generar eventos diarios, semanales, mensuales... mientras que el resto del tiempo, el procesador puede entrar en modos de bajo consumo y ahorrar batería.

Para cumplir su cometido, el RTC necesita:

- **Fuente de reloj de ultra bajo consumo:** Generalmente alimentado por un oscilador LSE de 32,768 kHz. Esta frecuencia no es casual: $32768 = 2^{15}$, lo que permite utilizar un divisor binario de 15 etapas para rebajar la frecuencia a 1 Hz (un segundo) con una corriente del orden de los nanoamperios.
- **Dominio de respaldo (*Backup Domain*):** El módulo RTC suele ubicarse en una zona del procesador con su propia alimentación, que no se desconecta aunque se apague el procesador. Esto permite que el calendario siga avanzando aunque el resto del procesador esté apagado. Adicionalmente, ciertas zonas de memoria también se encuentran en este dominio, para que al arrancar de nuevo el procesador, los datos no se hayan perdido.

Normalmente, el RTC lleva la cuenta del tiempo en segundos, minutos, horas, días y años, en lugar de tener un único contador global. Internamente gestiona las sumas utilizando BCD.

Además del seguimiento del tiempo pasivo, el RTC incluye funciones como alarmas para despertar al procesador de modos de bajo consumo profundo en fechas/horas programadas.

# 7. Periféricos y buses externos

Los procesadores tienen, por regla general, un gran número de pines disponibles para su conexión con dispositivos externos (de hecho, cuentan con varios miles en los modelos más avanzados). Sin embargo, esta disponibilidad tiene sus limitaciones: en primer lugar, no todos los pines son capaces de realizar todas las funciones. En segundo lugar, el uso de pines concretos requiere un grado de planificación del sistema completo a nivel de placa base.

Mucha veces querremos la posibilidad de conectar diferentes dispositivos a nuestro diseño, pero no necesariamente que éstos existan desde el comienzo. Por ejemplo, un microcontrolador sencillo puede utilizarse para controlar diferentes tipos de sensores, actuadores, pantallas, controles y, a nivel de diseño, sólo podremos ofrecer unos pocos pines al usuario.

Para tener la posibilidad de conectar multitud de dispositivos a nuestro procesador, existe el concepto de bus: protocolos que utilizan un conjunto reducido de pines, y permiten, a través de unas pocas líneas de datos, la interconexión de varios (en ocasiones hasta cientos) dispositivos. Dos de los protocolos más extendidos para este propósito son el **I2C** y **SPI**.

## 7.1. I2C

El **Bus I2C** (*Inter-Integrated Circuit* es una de estas alternativas. Es de las más sencillas, y permite, a través de únicamente dos líneas, la conexión de hasta 1000 dispositivos. Los elementos más destacables del bus I2C pueden verse en la Figura 8 y son:

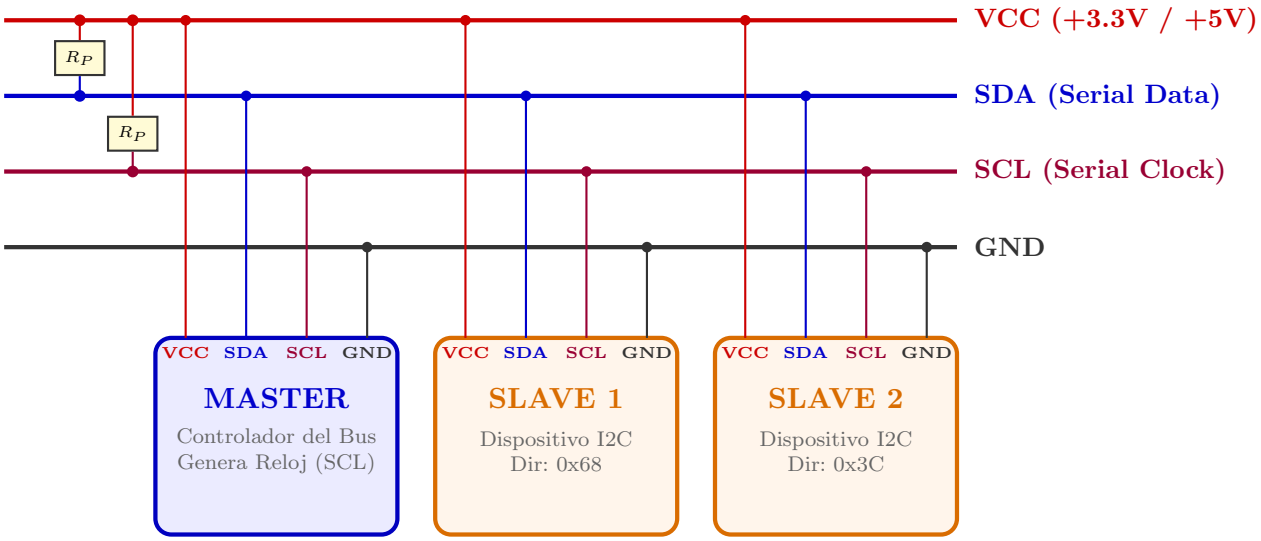



Figura 8: Diagrama de un bus I2C

- *Master*: Un único dispositivo que tiene el control del bus, y genera las peticiones en el mismo. El *Master* puede cambiar con ciertas señalizaciones especiales, pero siempre debe ser un único dispositivo.
- *Slave*: Pueden ser múltiples, son dispositivos que están conectados al bus y escuchan el mismo a la espera de peticiones del *Master*.
- **SDA** o *Serial Data*: Línea de datos serie, a través de la cual se envía la información (bien del *Master* al *Slave* o viceversa).

- **SCL** o *Serial Clock*: Línea de reloj serie. Controlada por el *Master*, lleva la señal de reloj con la que se sincronizan los datos de **SDA**. Generalmente, los dispositivos **I2C** permiten un amplio rango de frecuencias para su funcionamiento, aunque las más típicas son 100 o 400 KHz.

Un dato interesante a observar es el hecho de que tanto **SCL** como **SDA** tienen resistencia de *pull-up* a VCC. Es decir, que por defecto están siempre a "1". Cualquier dispositivo que se conecte, operará en modo de *colector abierto* (open drain). Es decir, que únicamente podrá bajar la línea a "0" conectándola a tierra, enviándose los "unos" pasivamente.

### 7.1.1. Funcionamiento del bus I2C - capa física

Una vez realizadas las conexiones entre los dispositivos, la siguiente pregunta razonable es cómo opera el bus I2C. Las transferencias de datos siguen, de manera general, los siguientes pasos:

1. Una transferencia se inicia con una condición de inicio (**S** (start)), que se señala desde el *Master* bajando **SDA** a "0", mientras **SCL** se mantiene a "1".
2. A continuación **SCL** comienza a oscilar, bajando a "0" en primer lugar, mientras se prepara el bit a enviar en **SDA**.
3. Cuando **SCL** sube a 1, el bit enviado está disponible en **SDA** y el resto de dispositivos *Slaves* pueden leerlo.
4. Se repiten los pasos 2 y 3 tantas veces sea necesario para enviar todos los bits.
5. El último bit es seguido de un pulso de reloj sin dato, en el cual **SDA** se pone a "0".
6. Finalmente, se indica la condición de fin (**P** (stop)), liberando primero **SCL**, y posteriormente **SDA**.

Este proceso se puede ver gráficamente en la Figura 9.

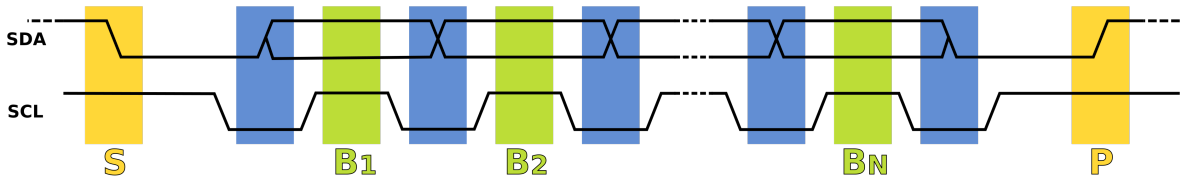



Figura 9: Transferencia de datos en I2C. (fuente:wikipedia)

### 7.1.2. Direccionamiento en I2C - capa lógica

Ahora bien, dentro de estas transferencias, ¿cómo sabemos a qué dispositivo van los bits? para ello, cada *Slave* cuenta con una dirección (de 7 bits, o 10 en versiones más modernas), habitualmente fijada por hardware, y con la que se puede identificar un dispositivo concreto. Así, para escribir, se seguirá el siguiente procedimiento:

1. Inicio de comunicación con condición inicial **S** seguida de la dirección (7 bits) + bit "0" (write). *Slave* responde **ACK**.
2. Envío del comando (o registro destino) (8 bits). *Slave* responde **ACK**.
3. Envío del dato (8 bits). *Slave* responde **ACK**.
4. Generalmente, y según dispositivo, se pueden enviar datos consecutivos que irán a parar a registros consecutivos si se quiere.
5. Fin de comunicación con condición de parada **P**.

Para leer, el proceso es algo más complejo, ya que primero indicaremos el registro en modo escritura, y posteriormente cederemos el control al *Slave* en modo lectura para que nos pase el dato por el bus.

1. Inicio de comunicación con condición inicial **S** seguida de la dirección (7 bits) + bit "0" (write). *Slave* responde **ACK**.
2. Envío del comando (o registro destino) (8 bits). *Slave* responde **ACK**.
3. Re-Inicio de comunicación con condición inicial **S** seguida de la dirección (7 bits) + bit "1" (read). *Slave* responde **ACK**.
4. El *Slave* toma el control y envía el dato de vuelta (8 bits). El *Master* puede responder con **ACK** para pedir más datos consecutivos, o con **NACK** para terminar la comunicación.
5. Fin de comunicación con condición de parada **P**.

Las condiciones **ACK** y **NACK** se indican, respectivamente, con un "0" y un "1" (sí, están invertidas) por parte del receptor, en la línea **SDA** tras las transmisión de un byte.

El proceso para ambas transacciones puede observarse en la Figura 10

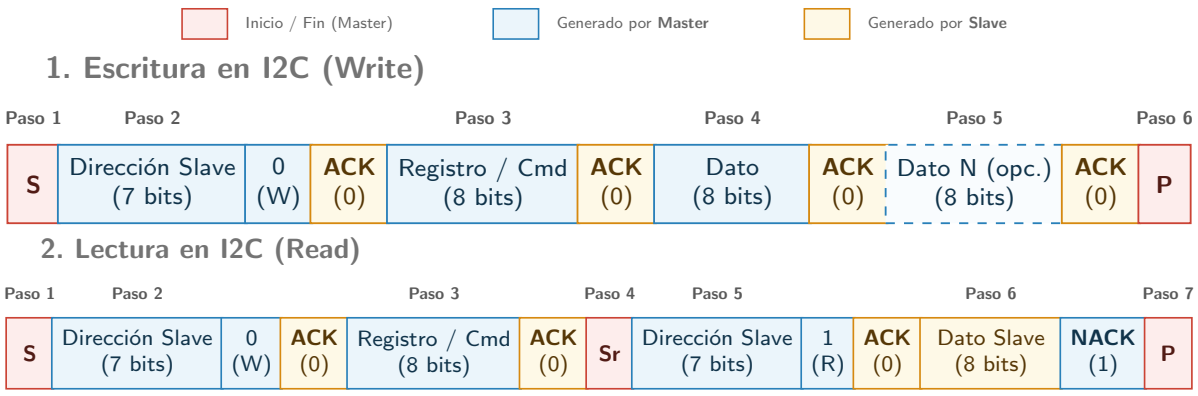



Figura 10: Escrituras y lecturas, respectivamente, en I2C

### 7.1.3. Líneas auxiliares a I2C

En ocasiones, utilizar únicamente las líneas **SCL** y **SDA** no es suficiente. Pensemos por ejemplo en el caso de las interrupciones. Con tan sólo las dos líneas de I2C, capturar una interrupción es imposible, tendríamos que hacer una encuesta activa a los periféricos, preguntando periódicamente si tienen algún dato de interés. Esto sería claramente ineficiente.

Es por ello que las interrupciones de los dispositivos sí van por líneas separadas, para que el procesador sea capaz de capturarlas, identificar directamente el dispositivo causante, y entonces, y sólo entonces, encuestarlo vía I2C para recibir el dato. Adicionalmente, algunos dispositivos disponen de otras señales extra, para por ejemplo habilitarlos o resetearlos.

## 7.2. SPI

El **Bus SPI** (*Serial Peripheral Interface*) es un protocolo de comunicación serie síncrono diseñado para la interconexión de componentes a corta distancia dentro de una misma placa (si bien también se utiliza entre placas cercanas). A diferencia de I2C, SPI es un bus *Full Duplex* (permite transmitir y recibir datos de manera simultánea) y no utiliza un direccionamiento por software, sino líneas físicas dedicadas para seleccionar cada periférico.

Los elementos principales del bus SPI y sus conexiones típicas se resumen en la Figura 11:

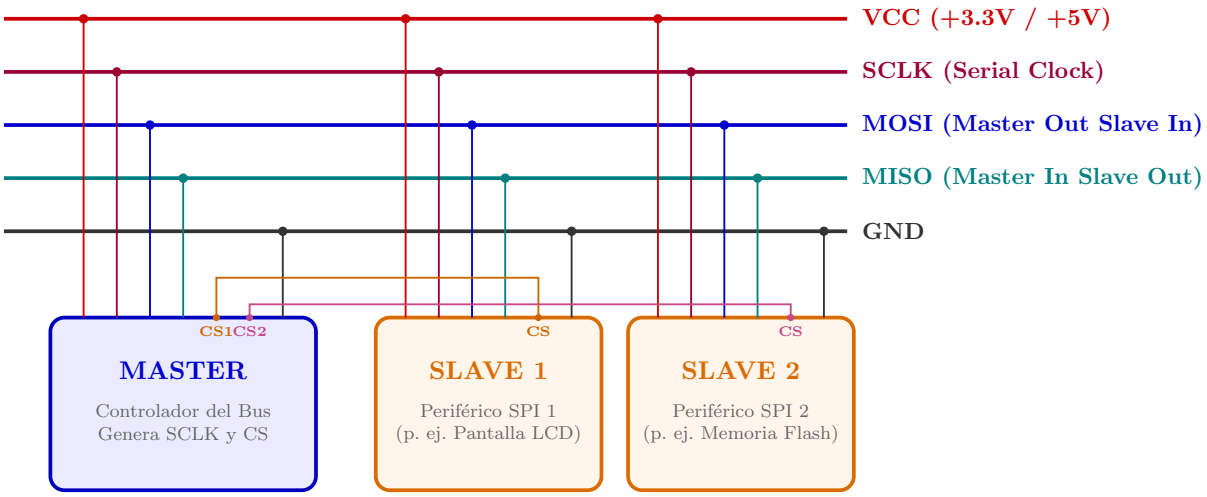



Figura 11: Diagrama de un bus SPI en configuración de selección independiente

- *Master*: Dispositivo controlador que inicia las transacciones, genera la señal de reloj principal (**SCLK**) y gestiona las líneas de selección de cada esclavo **CSx**.
- *Slave*: Dispositivos receptores/emisores. Únicamente están atentos a la información del bus cuando su señal individual de **CSx** está activa. Así se asegura exclusividad en las líneas del bus siempre y cuando el *Master* lo gestione de manera adecuada.
- **SCLK** (*Serial Clock*): Línea de reloj generada por el *Master*. A diferencia de **I2C**, **SPI** no define un límite estricto de velocidad y suele trabajar a frecuencias más altas (en el entorno de MHz).
- **MOSI** (*Master Out Slave In*): Línea serie para el envío de datos desde el *Master* hacia el *Slave*.

- **MISO** (*Master In Slave Out*): Línea serie para la recepción de datos en el *Master* provenientes del *Slave*.
- **CS** (*Chip Select*): Líneas dedicadas e independientes (activas a nivel bajo, "0") para habilitar el *Slave* concreto con el que se desea comunicar.

### 7.2.1. Funcionamiento del bus SPI - capa física

A diferencia de las líneas con *pull-up* de I2C, SPI utiliza salidas activas controladas por el *Master*. Esto elimina la necesidad de las resistencias, permitiendo velocidades mayores (debido a tiempos de conmutación más rápidos).

La información que atraviesa las líneas **MOSI** y **MISO** se almacena con registros de desplazamiento, en los que la unidad de transferencia básica es un byte completo.

Por cada pulso introducido en **SCLK**:

1. El *Master* desplaza un bit hacia el *Slave* por la línea **MOSI**.
2. Simultáneamente, el *Slave* desplaza un bit hacia el *Master* por la línea **MISO**.
3. Tras 8 ciclos de reloj, ambos dispositivos han transferido un byte completo.

**Modos de transmisión SPI (CPOL y CPHA):** Los dispositivos definen su modo de operación en función de dos parámetros configurables en el hardware:

- **CPOL** (*Clock Polarity*): Define el estado de reposo de la línea de reloj **SCLK**. Si CPOL = 0, la línea reposa en "0" lógico; si CPOL = 1, reposa en "1".
- **CPHA** (*Clock Phase*): Define en qué flanco del reloj se capturan los datos. Si CPHA = 0, los datos se muestrean en el primer flanco y se muestrean en el segundo; si CPHA = 1, al contrario.

La combinación de ambos parámetros da lugar a los 4 modos SPI estándar:

| Modo SPI | CPOL | CPHA | Reloj en reposo | Flanco de Muestreo |
|---|---|---|---|---|
| **Modo 0** | 0 | 0 | Nivel Bajo (0) | Flanco de subida |
| **Modo 1** | 0 | 1 | Nivel Bajo (0) | Flanco de bajada |
| **Modo 2** | 1 | 0 | Nivel Alto (1) | Flanco de bajada |
| **Modo 3** | 1 | 1 | Nivel Alto (1) | Flanco de subida |

Tabla 1: Modos de funcionamiento en el protocolo SPI

### 7.2.2. Funcionamiento del bus SPI - capa lógica

Una vez se transmite un byte, su significado no tiene una descripción en el estándar (como sí tiene **I2C**), y depende del fabricante del dispositivo el definir su comportamiento. Habitualmente, el primer byte enviado contendrá el comando que ejecutar o la dirección a la que escribir, mientras que los bytes siguientes serán los datos a transmitir. No obstante es importante recalcar que el funcionamiento específico dependerá del periférico conectado.

### 7.2.3. Topologías de conexión

A nivel de diseño de hardware, existen dos métodos principales para interconectar múltiples periféricos SPI a un mismo controlador:

1. (***Independent Slave / Parallel Configuration***)**:** Es la topología estándar vista en la Figura 11. Las líneas de reloj y datos son compartidas por todos los periféricos, pero el controlador necesita una línea **CSx** dedicada para cada esclavo.
2. (***Daisy Chain Configuration***)**:** En dispositivos diseñados específicamente para ello, la salida **MISO** de un esclavo se conecta a la entrada **MOSI** del siguiente. De este modo, una única línea **CS** común activa todos los esclavos a la vez. La información se pasa de uno a otro secuencialmente como un único gran registro de desplazamiento.

## 7.3. Dispositivos externos mapeados en memoria (FMC)

En los buses serie como **I2C** o **SPI**, los comandos y datos se envían de forma secuencial, a través de un número reducido de hilos. A nivel de software, esto se traduce en drivers que deben, poco a poco, enviar y recibir toda la información que se desee. Esto es notablemente más complicado que el trabajo con los periféricos integrados del procesador, cuyos registros se mapean directamente en memoria.

Para dispositivos externos con gran cantidad de registros (o directamente, memorias), los microcontroladores actuales suelen incorporar periféricos dedicados que permiten mapear, en el propio espacio de memoria del procesador, el mapa de memoria del dispositivo externo. Esto facilita enormemente la interacción. Por ejemplo, uno muy habitual es el `FMC` (*Flexible Memory Controller*).

Lo hacen conectando el bus interno del procesador directamente con un bus paralelo externo (bus de direcciones, bus de datos y líneas de control) mediante un controlador de memoria intermedio que realiza las traducciones de un espacio de direcciones al otro. Así, el acceso al dispositivo se realiza mediante instrucciones estándar de lectura y escritura en C, exactamente como si se tratara de la memoria RAM o Flash internas. Se muestra un esquema en la Figura 12:

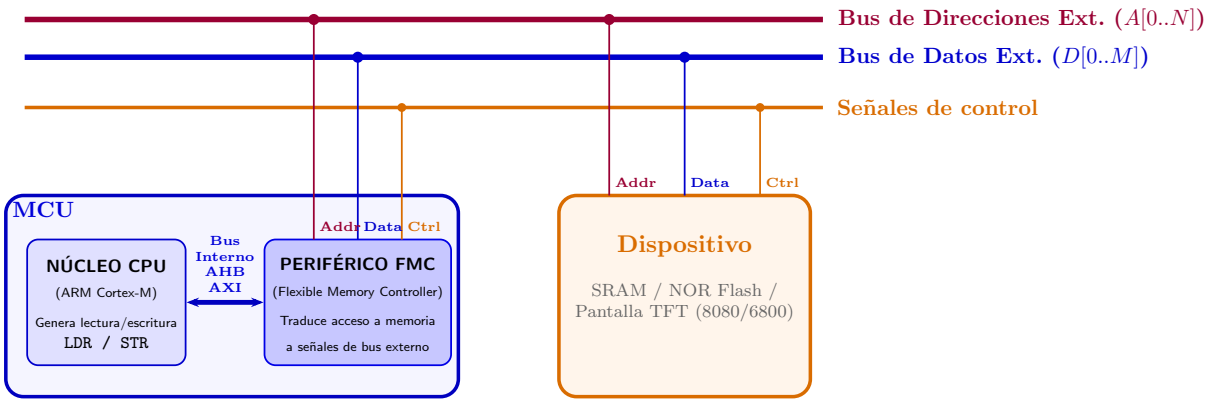



Figura 12: Interconexión paralela mediante controlador FMC con desglose del bus interno de sistema

- **Bus de Direcciones** ($A[0.,25]$)**:** Define la dirección física a la que se desea acceder. El espacio completo puede ocupar decenas de Megabytes.

- **Bus de Datos ($D[0.,15]$ o $D[0.,31]$):** Bus bidireccional de ancho variable. Permite transferir palabras completas en un único ciclo de reloj.
- **Señales de control** Señales variadas, dependientes de la tecnología a la que se conecta (Flash, SRAM, NOR, NAND), que indican la validez de datos/dirección, direccionalidad de la comunicación, activación de periféricos...

### 7.3.1. Acceso por software

La mayor ventaja del mapeo es su facilidad de uso desde software. Para escribir en un controlador de memoria externo basta con realizar asignaciones directas mediante punteros.

Listing 1: Ejemplo de acceso directo a memoria externa mapeada mediante FMC

```
// Direccion base asignada al Banco 1 de la memoria externa
#define EXT_RAM_ADDR  ((volatile uint16_t *) 0x60000000)

void escribir_datos_RAM(void) {
    // Escribe un dato directamente en la SRAM externa mediante el bus FMC
    *(EXT_RAM_ADDR + 0) = 0xABCD; // FMC envía el dato automáticamente
}   //NOTA: por supuesto el FMC debe estar configurado previamente
```

Un caso común de aplicación de controladores FMC es la gestión de pantallas. La pantalla expone una dirección o conjunto de ellas donde enviar la información sobre los píxeles o comandos. Desde el procesador, bastará con mapearlo internamente y escribir ahí, evitando gastar recursos de memoria internos para almacenar las imágenes o gráficos mostrados. Esto es especialmente relevante en microcontroladores con memoria reducida.

Adicionalmente, esto permite que los controladores DMA *Direct Memory Access* transfieran buffers enteros de imágenes directamente a la pantalla a velocidades de decenas de Megabytes por segundo sin intervención de la CPU, mediante ráfagas a los registros mapeados.

## 7.4. Comparativa

Cada una de las interfaces vistas cumple una función diferente dentro del procesador, pero en esencia todas ellas nos permiten la conexión con dispositivos externos. Un resumen de sus características se muestra en la Tabla 2.

| **Periférico** | **I2C** | **SPI** | **FMC** |
|---|---|---|---|
| **Nº de pines** | 2 | 4 | 20-60+ |
| **Conectividad** | Serie | Serie | Paralelo |
| **Velocidad** | 100 kHz – 3.4 MHz | 10 MHz – 100 MHz | 10 MHz – 100 MHz |
| **Ancho de Banda** | kb/s | Mb/s | MB/s |
| **Acceso** | Protocolo estándar (Comandos, ACK) | Protocolo específico (Intercambio de Bytes) | Escritura/Lectura mapeada en memoria |
| **Aplicación principal** | Sensores lentos | Sensores rápidos, memorias y pantallas de baja resolución | Memorias y pantallas de alta velocidad |

Tabla 2: Comparativa general

# 8. Multitarea

A menos que tengamos un código que sólo cumpla una función, habitualmente tendremos diversas tareas independientes que se tienen que completar en diferentes momentos, y que potencialmente intercambian información entre ellas. A grandes rasgos, se distinguen dos modelos de multitarea:

1. En un sistema guiado por **eventos**, existe una colección de funciones asociadas a un conjunto de eventos externos. La ocurrencia (asíncrona) de eventos determina el secuenciamiento de las funciones, que en caso de eventos simultáneos se ejecutan según prioridad. Los eventos pueden ser detectados mediante *polling* o mediante interrupciones.
2. En un sistema guiado por **tiempo**, las funciones se asocian con un conjunto de instantes de tiempo. En este caso, el transcurso (síncrono) del tiempo es quien determina el secuenciamiento de funciones, también con un atributo de prioridad para casos de ejecuciones simultáneas. El paso del tiempo se mide utilizando un temporizador, que interrumpe con una cierta periodicidad (ticker del sistema).

## 8.1. Arquitecturas multitarea cooperativa

Existen numerosas arquitecturas para la ejecución multitarea, que generalmente encuentran un compromiso entre simplicidad y capacidad.

### 8.1.1. Super-loop

Un único bucle infinito comprueba, en orden cíclico, el estado de los dispositivos de entrada/salida, y los sirve cuando sea necesario. En este modelo no hay interrupciones. El problema es que las latencias o tiempoes de respuesta son impredecibles, y en el peor caso implican la duración completa de una vuelta al bucle. Además, la arquitectura es frágil pues cualquier cambio en el código altera los tiempos, y el comportamiento del sistema es muy dependiente de cómo esté programada la secuencia de eventos. El esquema básico de Arduino, por ejemplo, funciona con esta arquitectura:

```
void init() {
    //configuración de registros, relojes, etc
    loop();
}

void loop() {
    while (1) {
        if (task1_ready)
            task1();
        if (task2_ready)
            task2();
        //...
    }
}
```

### 8.1.2. Foreground/Background

En esencia, un *super-loop* con interrupciones. Varias tareas atienden a los dispositivos de E/S, implementadas como RTI. Estas tareas únicamente marcan eventos, actualizan variables y leen o escriben datos en colas. Se ejecutan en *foreground*, ganando prioridad sobre la tarea principal, y pueden o no anidarse. La tarea principal, siempre en background, realiza un bucle infinito y tiene la mínima prioridad. Irá procesando, poco a poco (siguiendo la estructura del super-loop) las diversas variables que se hayan disparado en las RTI.

Tiene la problemática de ubicar el grueso del procesamiento de los dispositivos de E/S. Si se coloca este procesamiento en *background*, todos los dispositivos acabarían de facto teniendo la misma prioridad, con retrasos similares a super-loop en el peor de los casos. Si se hace en *foregrond*, las RTI se alargan y se penaliza el tiempo de respuesta.

```
void init() {
    //configuración de registros, relojes, etc
    //configuración de RTI
    loop();
}

void loop() {
    while (1) {
        if (flag_a) {
            flag_a = 0;
            task_a();
        }
        if (flag_b) {
            flag_b = 0;
            task_b();
        }
        //...
    }
}

void isr_a_handler() {
    //trata isr de periférico a
    flag_a = 1;
}

void isr_b_handler() {
    //trata isr de periférico b
    flag_b = 1;
}
```

### 8.1.3. Cola de funciones

Añade prioridad a la ejecución de funciones *foreground/background* mediante una pila de tareas pendientes. Esta pila se llena al ser disparadas las interrupciones. La tarea principal *main* está continuamente de nuevas tareas apareciendo en la pila, y las ejecuta según llegan, respetando por tanto el orden de interrupción, pero no necesariamente de prioridad en caso de que una tarea más prioritaria interrumpiera tras una menos prioritaria.

```
volatile queue_t queue;

void init() {
    //configuración de registros, relojes, etc
    //configuración de RTI
```

```
6       loop();
7   }
8
9   void loop() {
10      while (1) {
11          if (!queue.empty()) {
12              ptr_task = queue.dequeue();
13              (*ptr_task)();
14          }
15      }
16  }
17
18  void isr_a_handler() {
19      //trata isr de periférico a
20      if (!queue.full())
21          queue.enqueue(task_a);
22  }
23
24  void isr_b_handler() {
25      //trata isr de periférico b
26      if (!queue.full())
27          queue.enqueue(task_b);
28  }
```

## 8.2. Arquitecturas RTOS

En las diferentes arquitecturas de multitarea cooperativa, se presenta un problema común: cuando una tarea está en ejecución y se bloquea, siempre bloqueará al resto. Esto presenta un gran problema pues el sistema completo se cae debido a un único punto de fallo. Para solventarlo, se debe dejar de confiar en el aspecto *cooperativo*, y moverse hacia una arquitectura *expropiativa* capaz de retirar de ejecución las tareas. Esto se hace mediante la presencia de un Sistema Operativo en Tiempo Real o **RTOS**.

En él, varias tareas RTI atienden a los dispositivos de E/S en *foreground*, mientras que otras tareas en *background* realizan el resto de funciones, ya sean independientes o disparadas en algún momento por las RTI. No hay necesariamente tarea principal o *main*, que desaparece tras configurarse todo el sistema. Para comunicarse entre sí, las tareas utilizan variables compartidas, lo cual implica la necesidad de poder detectar y resolver el acceso a secciones críticas.

Existe un código de **kernel** que se encarga de orquestrar todas estas intercomunicaciones. Ofrece mecanismos para sincronizar y comunicar tareas, decide según prioridades la tarea en ejecución, y lo más importante, **conmuta** la ejecución de tareas dinámicamente. La única desventaja presente en este sistema es la sobrecarga de tener al propio kernel en ejecución, pero sus ventajas lo compensan ampliamente.

Aunque un RTOS puede ser cooperativo (donde una tarea cede voluntariamente el control al kernel, terminando o mediante señales específicas), lo más habitual es contar con un RTOS expropiativo, donde las excepciones (RTI o propias del kernel) disparan la conmutación de tareas. En este caso, ante los eventos:

- La tarea en ejecución se interrumpe y el kernel guarda su contexto.
- La RTI se ejecuta y potencialmente pide al kernel la ejecución de cierta tarea.
- El kernel marca como preparada la ejecución de la tarea que esperaba el dato.

- El planificador conmuta a la tarea más prioritaria en cada momento, que podría ser (o no) la nueva tarea encolada.

```
void main() {
    // crea tareas
    // initializa RTOS
}

void isr_a_handler() {
    //trata isr de periférico a
    signal(A);
}

void isr_b_handler() {
    //trata isr de periférico b
    signal(B);
}

void task_a() {
    while (1) {
        wait(A);
        fun_a();
    }
}

void task_b() {
    while (1) {
        wait(B);
        fun_b();
    }
}
```

Para la sincronización entre las diferentes tareas (en *foreground* o *background*) se cuenta con mecanismos ofrecidos por el kernel como semáforos, mutex, variables compartidas... propios de un sistema operativo.

Cuando los datos se comparten entre diversas tareas, existe el problema de la desincronización cuando, por ejemplo, una tarea es interrumpida mientras está consultando los datos proporcionados por otra:

```
volatile uint16_t temp[2];
void leerTemperatura_isr(void) {
    // borraflag de int. pendiente
    temp[0] = // ...leer del HW...
    temp[1] = // ...leer del HW...
}

void main(void) {
    uint16_t0, t1;
    while(1) {
        // <--- INT: La RTI lee dos valores iguales (7 y 7 por ejemplo)
        t0 = temp[0];
        // <--- INT: La RTI lee dos valores iguales (8 y 8 por ejemplo)
        t1 = temp[1];
        // Sin embargo, la alarma suena al haberse guardado en momentos
            diferentes
        if( t0 != t1 )
            //suena alarma
    }
}
```

La solución pasa por hacer accesos atómicos a los datos compartidos, deshabilitando las interrupciones al acceder a los datos:

```
while(1) {
    INT_DISABLE();
    t0 = temp[0];
    t1 = temp[1];
    INT_ENABLE();
    if( t0 != t1 )
        //suena alarma
}
```

Aún así, este mecanismo no es trivial. Hay que asegurarse que toda inactivación es seguida por una activación, y que no se anidan parejas de inactivación/activación, ya que podrían activarse accidentalmente antes de tiempo.

Las interrupciones son, con todo, la forma idónea de dar respuesta rápida a eventos. Su latencia depende de:

1. El tiempo máximo en que están deshabilitadas
2. El tiempo que tarda en ejecutarse cualquier RTI de mayor prioridad (en caso de interrupciones anidables).
3. El tiempo que tarda el procesador en detectar la interrupción y saltar a la RTI.
4. El tiempo que se tarda en guardar el contexto y realizar la acción de respuesta.

El código por tanto deberá ser pequeño y eficiente para reducir los tiempos de ejecución y cambio de contexto, las secciones críticas cortas para reducir el tiempo de deshabilitación, y la arquitectura eficiente para detectar y saltar rápido.

## 8.3. Correción en aplicaciones multihebra

En una aplicación multihebra (por hebra entendemos un hilo de ejecución, ya sean tareas, interrupciones, etc), la correción debe garantizarse para evitar comportamientos erróneos (bloqueos principalmente).

Por un lado, los accesos a secciones críticas (zonas del código donde 2 o más hebras ueden acceder a la vez a un recurso compartido) deben ser atómicos. Es decir, sólo una de las dos puede estar en esa zona del código a la vez, para evitar que los valores se puedan consultar o escribir de manera errónea. Además, una vez se comienza un código atómico, se debe poder completar sin interrupción de otra hebra que acceda al mismo recurso.

Por otro lado, el código debe diseñarse para ser *reentrante*, esto es, que las variables locales se almacenen en pila y las globales se accedan de manera atómica, permitiendo así la ejecución por parte de varias hebra del mismo código.

Las secciones críticas siguen 3 principales patrones:

- *Read-modify-write*: Se lee un valor que se copia localmente, se modifica, y finalmente se actualiza la variable global que lo contiene.

- *Write-read*: Se escribe primero un valor, y se lee esperando que siga presente.
- *Multistep-write*: Se realizan varias escrituras consecutivas a memoria para valores que con una única instrucción no se actualizan (por ejemplo, estructuras complejas).

En arquitecturas super-loop, por ejemplo, no hay secciones críticas, pues solo existe una hebra. En arquitecturas foreground/background la hebra principal o `main` deberá proteger las secciones críticas inhabilitando las interrupciones. En un RTOS cooperativo, el programador controla cuándo cambia el contexto y por tanto podrá proteger las secciones críticas. En un RTOS expropiativo, no se tiene el control sobre los cambios de contexto y todas las hebras deberán proteger las secciones críticas de manera homogénea.

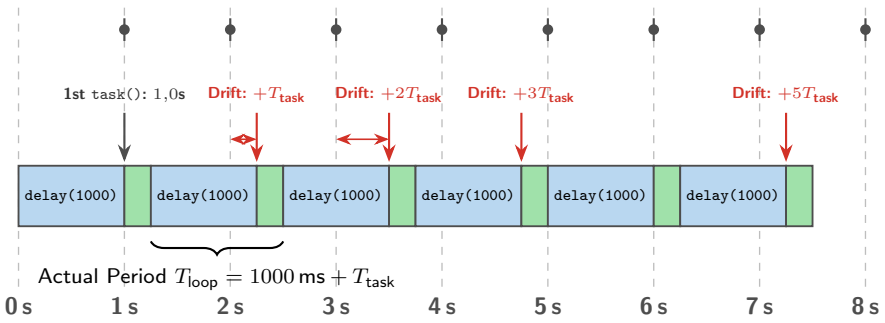
## 8.4. Sistemas guiados por tiempo

Existen gran número de sistemas empotrados que deben soportar la llamada periódica a determinadas funciones. Por ejemplo al refrescar un display a un cierto número de *frames* por segundo, muestrear sensores a una frecuencia concreta, etc. También es común que se realicen secuencias de acciones con cierta temporización, como por ejemplo en sistemas complejos que deben abrir/cerrar válvulas, mover motores, etc.

### 8.4.1. Primera aproximación

Una aproximación mediante sistemas *super-loop* nos llevaría al siguiente modelo de temporización, que falla por motivos obvios al no tenerse en cuenta el propio tiempo de ejecución de la tarea para separar las ejecuciones. Además, la CPU está activa todo el tiempo al realizar esperas activas:

```
void task_init( void );
void task( void );
void delay( uint16 ms );
void main( void ) {
    sys_init();
    task_init();
    while( 1 ) {
        delay_ms( 1000 );
        task();
    };
}
```



### 8.4.2. Segunda aproximación

Otra aproximación, que podría mejorar a la anterior, y evita tener a la CPU dormida, es tener un timer que se dispare cada cierto tiempo. Por ejemplo:

```
timer0_isr() {
    // borrar bits de ISR pendiente
    // no hace nada más, sólo saca del sleep();
}

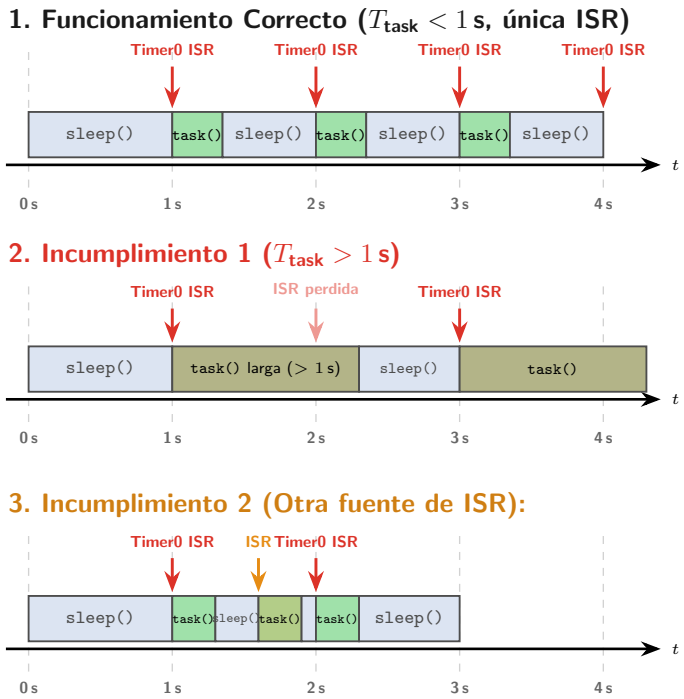
void main(void) {
    SET_TIMER_MS(timer0, 1000);

    while( 1 ) {
        sleep(); //entra en modo de sleep, se despierta con ISR
        task();  //realiza la tarea
    }
}
```

Esta aproximación funcionará mientras:

1. El tiempo de `task()` sea inferior siempre a 1 segundo.
2. Sólo exista una única fuente de interrupción.

Si se inclumple la primera, la tarea no se disparará el número de veces adecuado, saltando ciclos. Si se incumple la segunda, el problema vendrá porque otras fuentes de interrupción hagan que la llamada a `sleep()` acabe antes de tiempo, disparándose en más momentos que sólo cada segundo.



### 8.4.3. Tercera aproximación

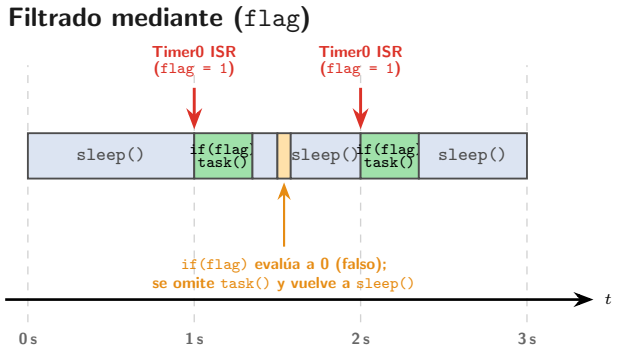
Una opción que sí sería funcional implicaría ejecutar la tarea en la propia ISR. Esto no es del todo recomendable, ya que las ISRs se deben mantener lo más escuetas posibles. Aunque no podemos solucionar el problema de que la tarea dure más de un segundo, sí podemos resolver el problema de que se lancen varias ISRs diferentes, mediante la inclusión de un flag que nos indique, al despertar del `sleep()`, si queremos de verdad ejecutar la tarea.

```
uint32_t flag = 0;

timer0_isr() {
    // borrar bits de ISR pendiente
    flag = 1;
}

void main(void) {
    SET_TIMER_MS(timer0, 1000);

    while( 1 ) {
        sleep(); //entra en modo de sleep, se despierta con ISR
        if (flag) {     //si flag está a 0, volverá a sleep()
            flag = 0;   //marca flag a 0
            task();     //realiza la tarea
        }
    }
}
```




Con este sistema, se pueden tener incluso varias tareas, disparadas por varios timers. También que un mismo timer, en función de una cuenta del número de disparos, active uno u otro flag para diferentes tareas. Mientras los tiempos de ejecución sean deterministas, e inferiores al periodo de disparo de el/los timer/s, cabrán en los mismos y el código funcionará correctamente.

### 8.4.4. Polling con espera en tareas periódicas

Una cuestión importante, y que hay que evitar en cualquier tipo de ejecución de tareas periódicas, es el polling con espera activa (por ejemplo esperando a que un periférico esté activo). Ya que, si el periférico nunca se activa, o tiene un error, nos quedaremos bloqueados esperando a que devuelva un control que nunca volverá.

Para solucionar este problema, se puede utilizar polling con timeout (dando un error tras un cierto tiempo), polling con consulta (si no está disponible, devuelve inmediatamente un error),

o un polling ciego (sin comprobación) si se tiene la certeza de que el dato está disponible (por ejemplo en sistemas sincronizados, aunque no es recomendable).

## 8.5. Planificadores cooperativos

Cuando se dispone de múltiples tareas, una primera aproximación puede incluir el preprogramar su orden de ejecución mediante una lista que se ejecuta periódicamente. Sin embargo, estos sistemas (llamados *cyclic-executive*) son muy poco flexibles, especialmente cuando se interactúa con personas o entornos que pueden generar interrupciones en momentos arbitrarios.

Como solución, se introduce el concepto de planificadores. En las Apartados 8.1 y 8.2 hemos visto diferentes ideas de cómo varias tareas pueden ejecutarse, pero no hemos explicado el concepto de prioridades entre ellas, y cómo un planificador puede (y debe) evaluar entre las tareas disponibles, cuál se debería ejecutar (basándose en prioridades, deadlines de ejecución, etc).

En todo momento existe una úncia tarea en ejecución, y se ejecutan hasta completarse, sin ser interrumpidas por otras. Las tareas se preparan con periodicidad fija (p.ej disparándose con un timer), se encolan en una lista de tareas preparadas y, cuando la CPU está libre, elige entre las mismas siguiendo algún criterio (puede ser simplemente el orden de cola, o algo más complejo basándose en prioridades).

Se necesita una fuente de interrupción periódica que dispara la ejecución de las tareas y, opcionalmente, pueden existir otras interrupciones que, en caso de poder ejecutarse simultáneamente, deberán proteger sus recursos mediante exclusión mutua.

Este tipo de planificador es muy simple, y la ejecución de tareas es predecible y fiable. Los recursos habitualmente no serán compartidos, por lo que no suele haber muchas secciones críticas. Sin embargo, tareas poco prioritarias, o eventos externos, pueden sufrir retrasos grandes debido a que la única forma de cambiar de tarea, es que la tarea actual termine su ejecución.

Las tareas se suelen definir por una estructura como la siguiente, llamada bloque de control de tarea:

```
typedef struct {
    void (* pfunction) (void);  //puntero a función ejecutora
    uint32 period; //periodo entre 2 ejecuciones
    uint32 ticks;  //cuenta de ticks desde la última ejecución
    boolean ready; //1 si está lista para ejecución
} task_t;

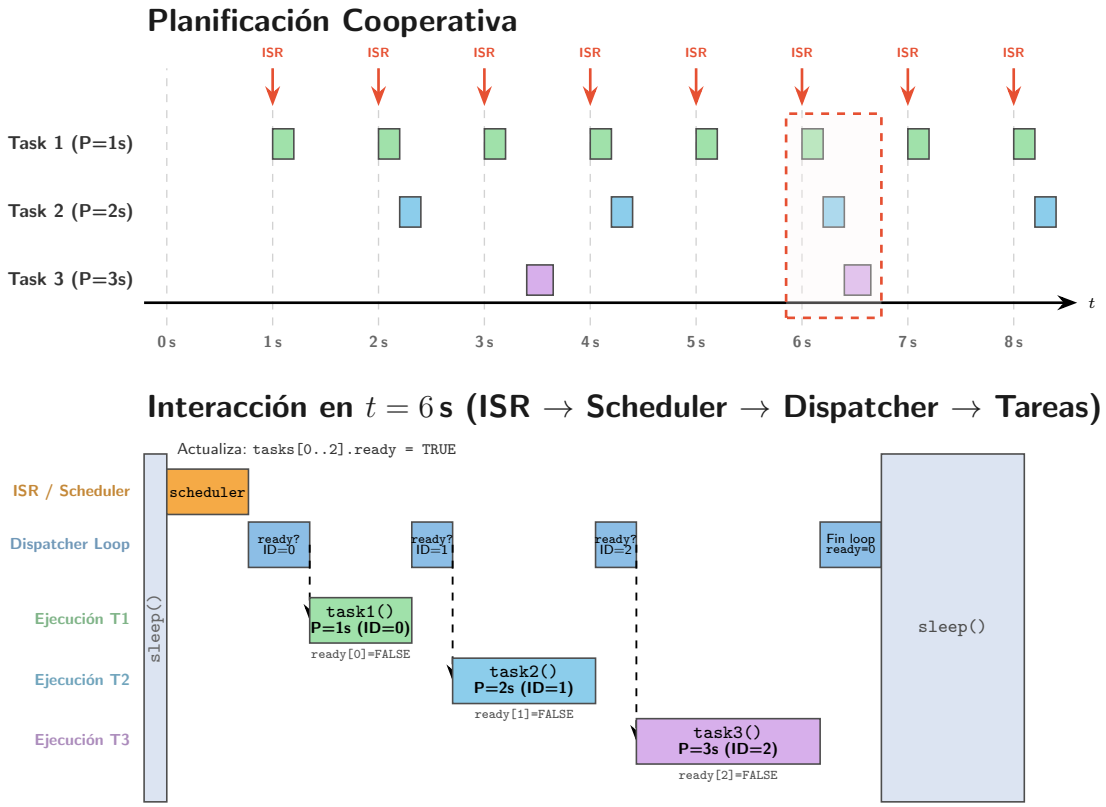
#define MAX_TASKS (10)
task_t tasks[MAX_TASKS];
```

A su vez, la gestión se divide en un planificador (que marca las tareas como listas), y un despachador o lanzador que las ejecuta:

```
//la rutina del timer es quien, de cuando en cuando,
//evalúa la planificación de tareas
timer0_isr() {
    // borrar bits de ISR pendiente
    scheduler();
}

void scheduler (void) {
```

```
9       uint32 id;
10      for (id=0; id<MAX_TASKS; id++)
11          if (tasks[id].pfunction)
12              if (++tasks[id].ticks == tasks[id].period) {
13                  //si una tarea llega a su periodo, se marca como READY
14                  //pero nunca expropia a otra (tras la ISR se volverá
15                  //a la ejecución en curso)
16                  tasks[id].ticks = 0;
17                  tasks[id].ready = TRUE;
18              };
19  }
20
21  void dispacher( void ) {
22      uint32 id;
23      for (id=0; id<MAX_TASKS; id++)
24          //el dispatcher ejecuta las tareas que están
25          //ready, una tras otra
26          if (tasks[id].ready == TRUE) {
27              (*tasks[id].pfunction)();
28              tasks[id].ready = FALSE;
29          }
30  }
31
32  void main (void) {
33      sys_init();
34      scheduler_init();
35
36      TIMER_INIT(timer0, 1000);
37      while (1) {
38          if (!tasks_ready()) //duerme hasta que interrumpa el scheduler
39              sleep();
40          //tras lo cual puede haber tareas listas para despachar
41          dispacher();
42          //si no las hubiera, o tras ejecutar, vuelvo a dormir
43      }
44  }
```




El modelo de planificador cooperativo se puede hacer más completo indicando, por ejemplo, prioridades, tiempos de activación de tareas, y multitud de parámetros adicionales para hacerlo más complejo. Cada uno tendrá sus ventajas e inconvenientes. Lo más importante es conocer que, en cualquier caso, la distribución óptima de tareas es un problema NP-complejo. Por muy inteligente que hagamos nuestro planificador, en sistemas con una gran contingencia entre tareas, lo más que podremos hacer es aproximarnos a una ejecución ideal de las tareas. Es por ello muy importante contar con las herramientas de perfilado adecuadas como para asegurar que el sistema cumple con los requisitos establecidos.

### 8.5.1. Bloqueos

Otro aspecto importante en este tipo de sistemas es la capacidad de recuperarse ante bloqueos. Si una tarea se queda atascada, el dispatcher no puede por sí mismo pararla. Para ello, se pueden introducir *watchdogs*. Un *watchdog* es un temporizador que, en caso de llegar a su límite, resetea el sistema a un cierto estado. Se puede, por ejemplo, en cada iteración del dispatcher, resetear el temporizador. Así, si se deja de resetear (por un bloqueo de alguna tarea) el temporizador llegará a su límite y reiniciará el sistema. La gestión se puede hacer de la siguiente manera:

```
1
2               void wd_isr() {
3                   SYSTEM_RESET();
4               }
5
6               static uint16 timeout;
7
8               void wd_on (uint16 dms) {
9                   timeout = dms;
10                  WTCNT = dms;
```

```
11                      //configuramos el watchdog para generar interrupciones al
                            llegar a
12                      //0 (cuenta descendente). Si llega, reiniciará el sistema
13                  }
14
15                  void wd_kick (void) {
16                      //reinicia la cuenta regresiva
17                      WTCNT = timeout;
18                  }
19
20                  void dispacher( void ) {
21                      //funcionalidad de dispatcher habitual
22                      //si se queda atascado, no llega nunca aquí,
23                      //y por tanto no reinicia el watchdog
24                      wd_kick();
25                      //también es posible crear una tarea wd_kick()
26                      //que vaya en la propia lista del dispatcher a máxima
                            prioridad
27                      //en caso de sistemas más complejos
28                  }
```

Al reiniciarse el sistema, se pueden consultar registros internos del procesador, o valores en memoria, para saber las causas (por ejemplo, si así lo preparamos y programamos, conocer la tarea causante del bloqueo).